

LAB 01: Packet Capture Fundamentals

Understanding network communication through packet-level analysis

Thursday, July 16, 2026 |

Table of Contents

1. Objective:	5
2. Lab Environment:	5
2.1: Host Machine:	5
2.2: Hypervisor:	5
2.3: Kali Linux:	5
2.4: Software:	5
3. Networking Concepts Primer:	6
3.1: What is a Packet?	6
3.2: The Wireshark Interface:	6
3.3: Display Filters:	7
4. Methodology	7
5. Evidence Collection Policy	8
6. Task 1: Interface Selection	8
6.1: Objective:	8
6.2 Architecture Diagram:	8
6.3 Procedure:	9
6.4 Screenshots:	9
6.5 Observation:	11
7. Task 2: Idle Capture Baseline	11
7.1: Objective:	11
7.2 Procedure:	11
7.3 Result:	11
7.4 Screenshots:	12
7.5 Analysis:	13
7.6 Conclusion:	13
8. Task 3: Traffic Generation (ICMP + DNS)	13
8.1: Objective:	13
8.2: Procedure:	13
8.3: Result:	13
8.4: Screenshots:	14

8.5: Analysis:	17
8.6: Conclusion:	17
9. Task 4: ICMP Filtering	17
9.1: Objective:	17
9.2: Procedure:	17
9.3: Result:	17
9.4: Screenshots:	18
9.5: Analysis:	19
9.6: Conclusion:	19
10. Task 5: DNS Resolution	19
10.1: Objective:	19
10.2: Procedure:	19
10.3: Result:	19
10.4: Screenshots:	20
10.5: Analysis:	22
10.6: Conclusion:	22
11. Task 6: HTTPS/TLS Capture	23
11.1: Objective:	23
11.2: Procedure:	23
11.3: Result:	23
11.4: Screenshots:	24
11.5: Analysis:	28
11.6 Conclusion:	28
12. Task 7: Packet Inspection	28
12.1: Objective:	28
12.2: Procedure:	28
12.3: Result:	28
12.4: Screenshots:	29
12.5: Analysis:	30
12.6: Conclusion:	30
13. Task 8: TCP Stream Reconstruction	30

13.1: Objective:	30
13.2: Procedure:	30
13.3: Result:	31
13.4: Screenshots:	31
13.6: Conclusion:	31
14. Task 9: Evidence Export	32
14.1: Objective:	32
14.2: Procedure:	32
14.3: Result:	32
14.4: Analysis:	32
14.5: Conclusion:	32
15. Overall Comparative Summary Table	32
16. Overall Conclusion	34
17. References	34

1. Objective:

This lab investigates how network traffic appears at the packet-level using Wireshark by capturing and analyzing traffic generated by common networking tools such as `ping`, `dig`, and `curl`. Multiple protocols, including ICMP, DNS, TCP, and TLS, are examined individually to understand how each appears on the network. Evidence is preserved through exported packet captures (`.pcapng`) and `tshark` output alongside annotated Wireshark screenshots, ensuring that all observations are supported by reproducible packet-level evidence rather than visual inspection alone.

The primary objective is to develop the packet analysis skills required for subsequent networking and cybersecurity labs. Future exercises involving network reconnaissance, port scanning, troubleshooting, incident investigation, and traffic analysis all rely on the ability to accurately interpret packet-level data. Packets represent the ground truth of network communication, whereas logs, alerts, and application output may be incomplete, misleading, or intentionally manipulated. Building this foundation enables later labs to move beyond simply running tools toward understanding the underlying network behavior that those tools reveal.

2. Lab Environment:

2.1: Host Machine:

- OS: Windows 11
- CPU: 13th Gen Intel Core i7-1355U, 1.70 GHz
- RAM: 8.00 GB

2.2: Hypervisor:

- Oracle VirtualBox 7.2.12

2.3: Kali Linux:

- Version: kali-linux-2026.2-virtualbox-amd64
- RAM Allocated: 2048 MB
- CPUs Allocated: 2
- Network Adapter: NAT Network (NatNetwork, 10.0.2.0/24)

2.4: Software:

- Wireshark 4.6.6 (GUI packet capture and analysis)
- `tshark` (command-line packet capture/export, bundled with Wireshark)
- Standard Linux networking utilities: `ping`, `dig`, `curl`

3. Networking Concepts Primer:

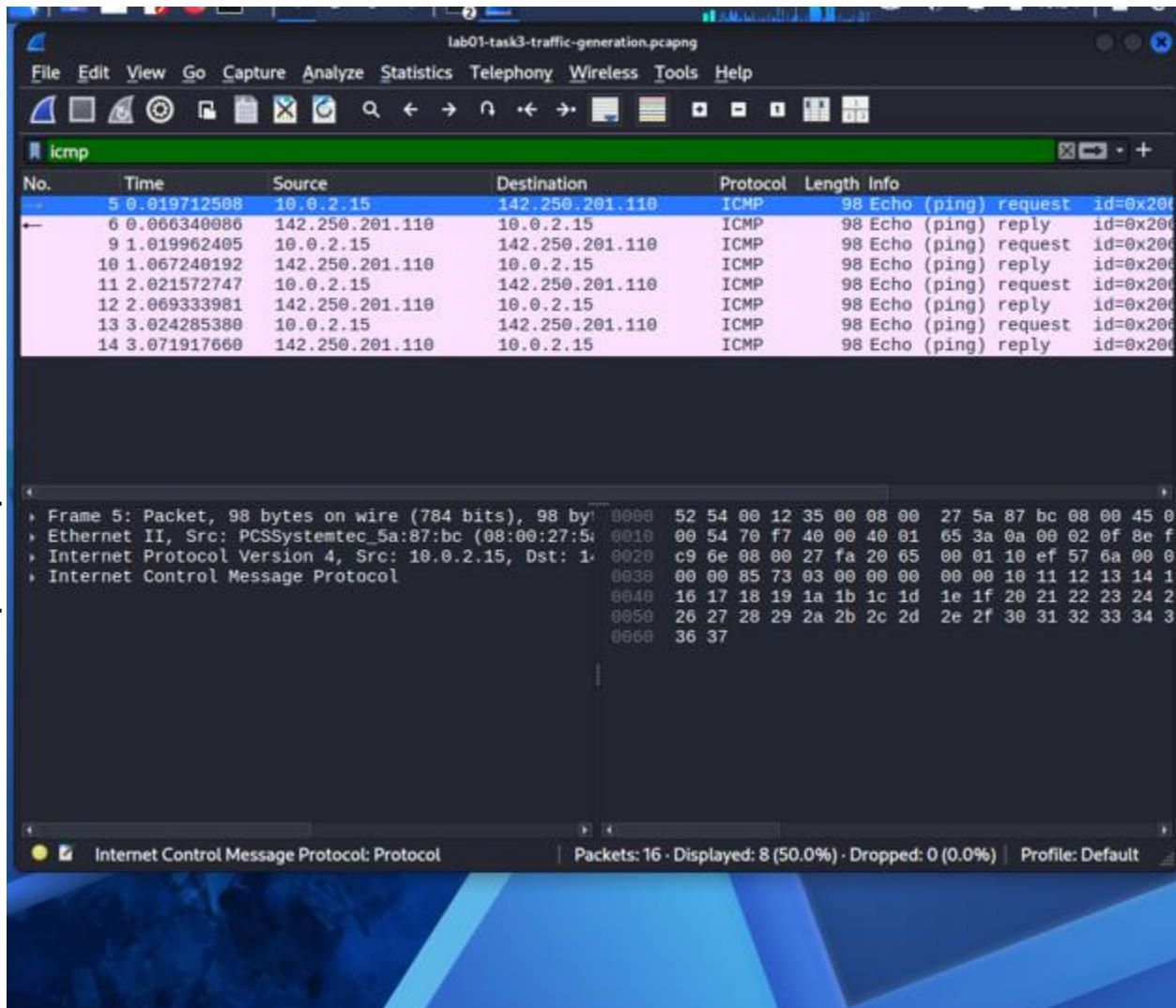
Foundational networking concepts, including 'ip a' output, MAC addressing, IP addressing, and CIDR/Subnet notation, were included in **Lab 00** and are not repeated here. This section focuses on the concepts required for packet capture and analysis.

3.1: What is a Packet?

Packet is a small unit of data transmitted across a network. It consists of protocol headers and a payload. The headers contain metadata such as source and destination addresses, protocol information, and other control fields, while the payload carries the actual application data. A useful analogy is a postal letter: the envelope contains the addressing information required for delivery, while the letter inside contains the message itself.

3.2: The Wireshark Interface:

Wireshark presents captured traffic using three primary panes:



1. **Packet List:** Displays a summary of every captured packet, including the timestamp, source, destination, protocol, length, and brief description.
2. **Packet Detail:** Provides a hierarchical, layer-by-layer breakdown of the selected packet, allowing inspection of Ethernet, IP, TCP/UDP, and application-layer protocol fields.
3. **Packet Bytes:** Displays the raw packets contents in hexadecimal and ASCII format, representing the exact bytes transmitted across the network.

3.3: Display Filters:

Display filters limit which packets are shown in the Packet List without modifying the underlying capture. For example, Filters such as icmp, dns or tcp display only packets matching those protocols while the complete capture remains intact. This distinction is important for forensic and incident response work because the original .pcapng file always preserves all captured traffic, allowing different filters to be applied throughout the analysis without altering or losing evidence.

4. Methodology

To ensure consistency and reproducibility throughout the lab, the same workflow was followed for each protocol under investigation.

1. A live packet capture was started in Wireshark on the '**eth0**' interface, previously verified in Lab 00 as the VM's NAT Network interface.
2. Network traffic was generated using a protocol-specific tool (ping, dig or curl)
3. The capture was stopped immediately after the required traffic had been generated.
4. The complete capture was saved as a .pcapng file before any analysis or filtering to preserve the original evidence.
5. Each capture was exported to a text file using tshark, providing a command-line representation of the recorded packets alongside the graphical Wireshark analysis.
6. Appropriate display filters (e.g. icmp, dns and tcp) were applied to isolate the relevant traffic for inspection.
7. Individual packets were examined through Wireshark's **Packet Details** pane to analyze addressing information, protocol headers, TTL values, and protocol-specific fields.
8. Where applicable, TCP conversations were reconstructed using Follow TCP Stream to examine the complete client-server exchange.

This workflow was repeated independently for each protocol and task. Rather than combining all traffic into a single capture, separate .pcapng and corresponding tshark output files were produced for every experiment. This approach simplifies analysis, improves reproducibility, and allows each protocol to be examined and referenced independently.

5. Evidence Collection Policy

Unless otherwise stated, every experiment in this lab includes the following evidence:

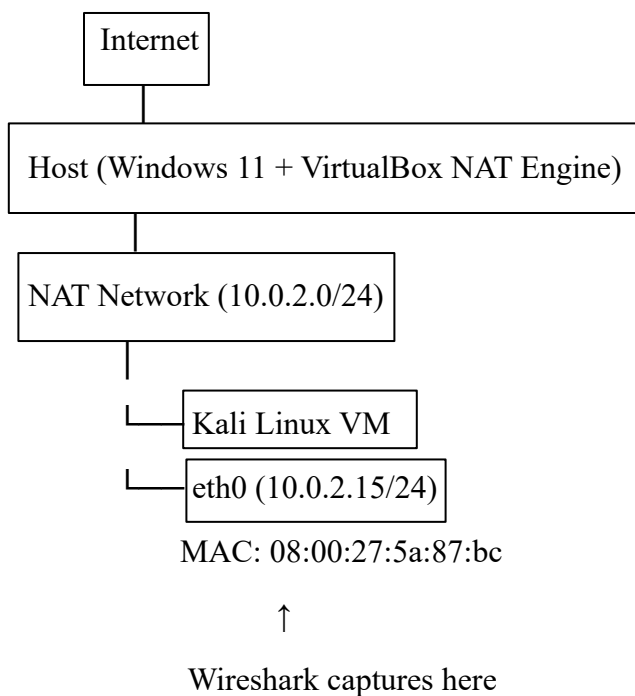
- Original, unfiltered .pcapng capture
- tshark text export of the capture
- Annotated Wireshark screenshots (interface view and/or filtered packet list)
- Exact commands executed to generate traffic
- Written observations and analysis of the captured behavior

6. Task 1: Interface Selection

6.1: Objective:

Identify and select the correct network interface for packet capture, corresponding to the VM's NAT Network connection.

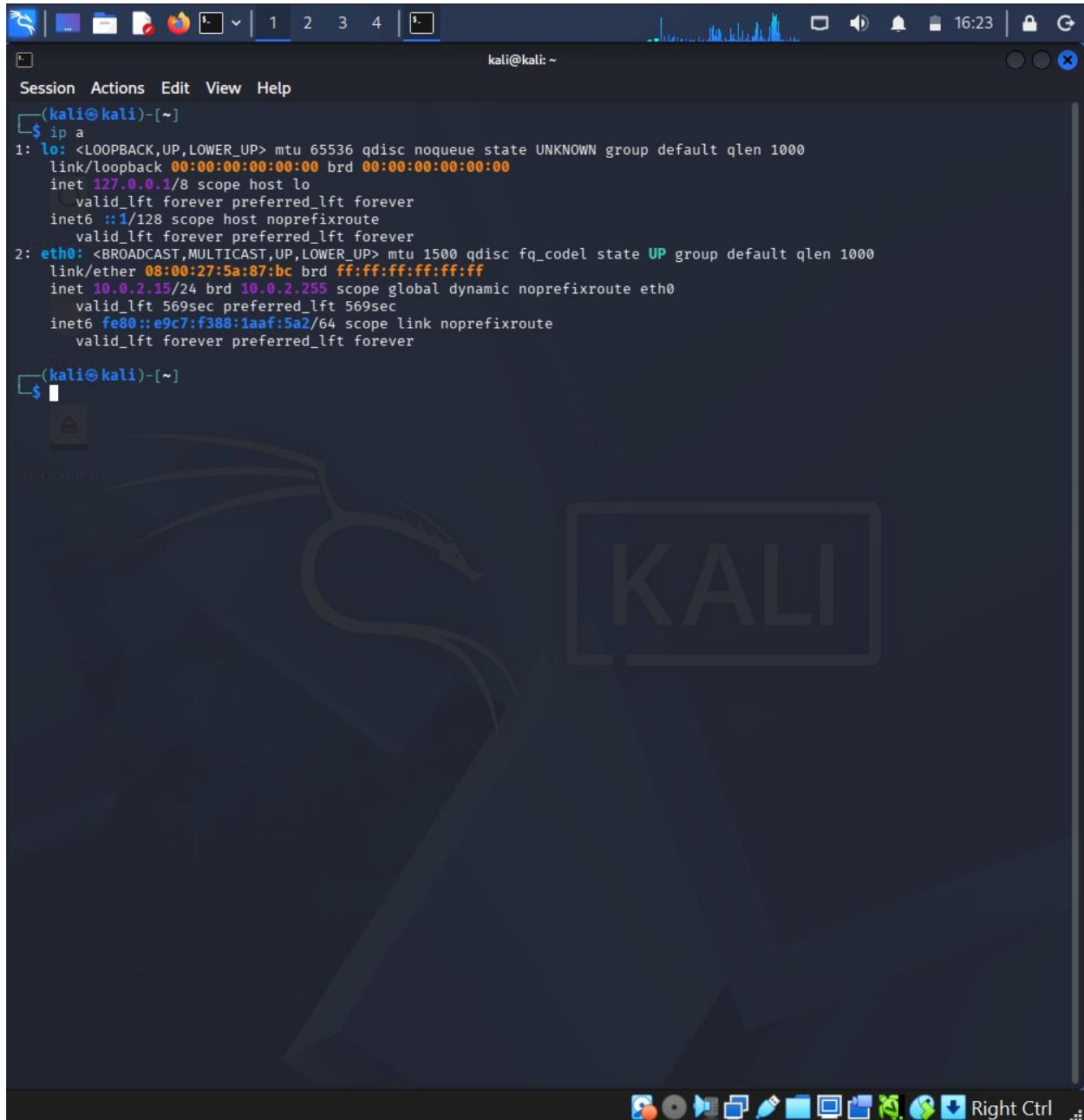
6.2 Architecture Diagram:



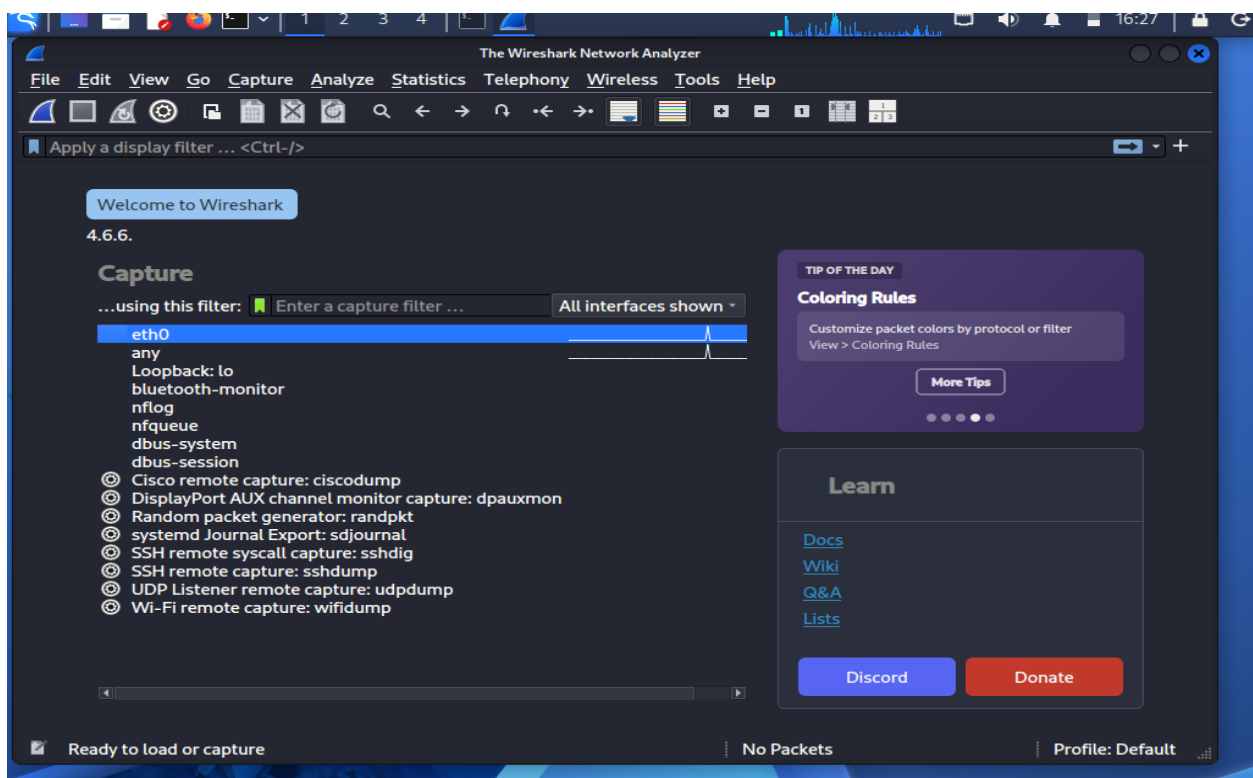
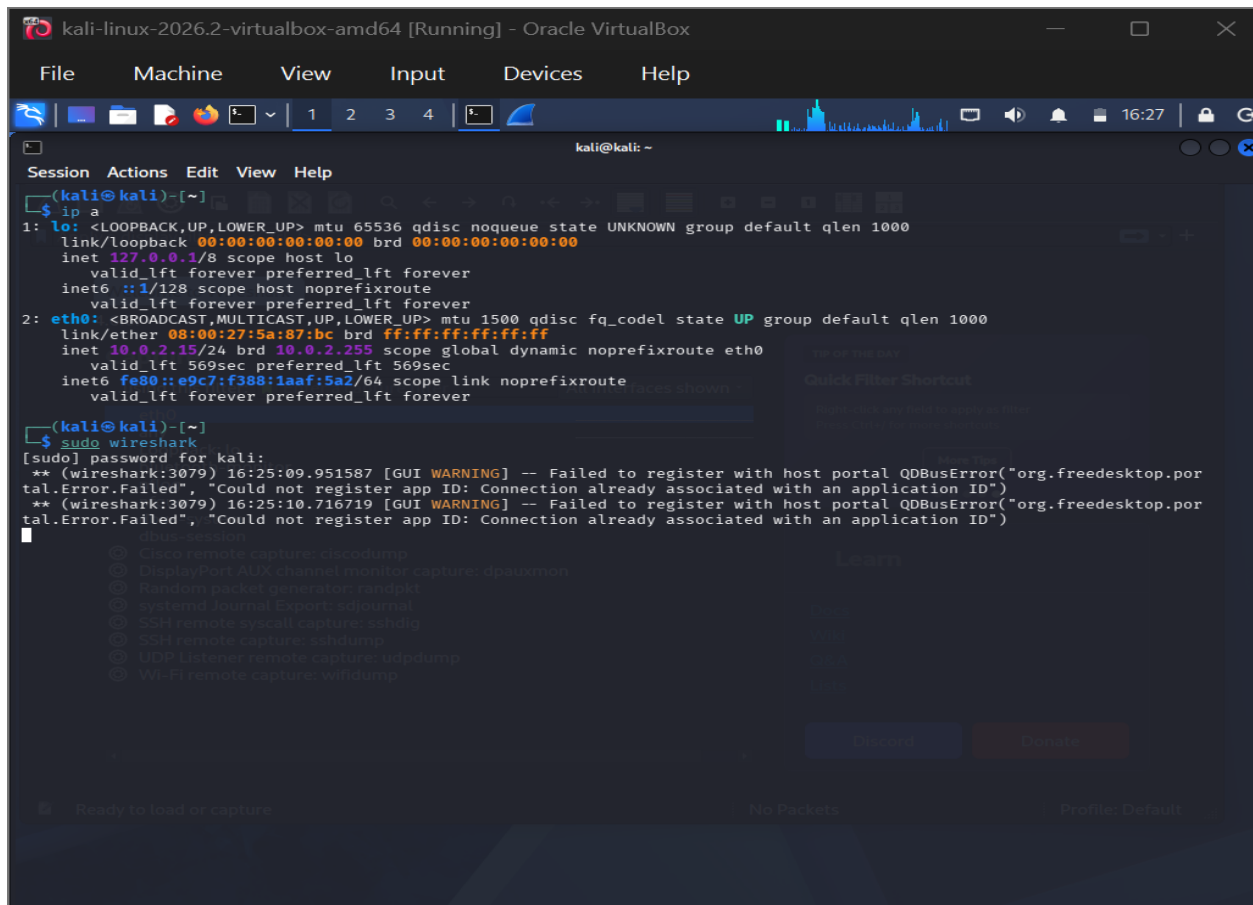
6.3 Procedure:

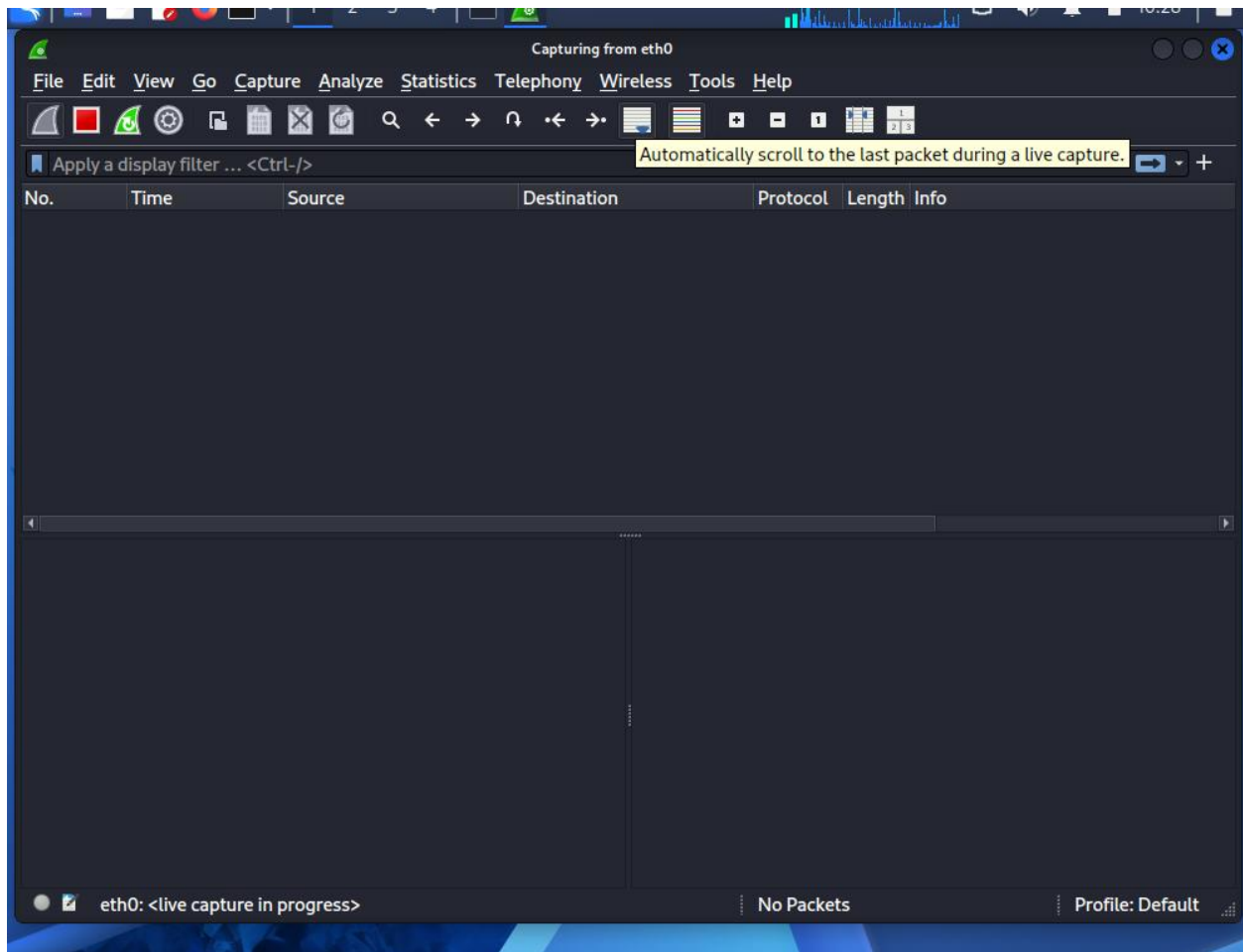
Before starting any capture, the active interface was confirmed using `ip a` in the Kali terminal. This showed `eth0` in an UP state with IP address `10.0.2.15/24` and MAC address `08:00:27:5a:87:bc`, consistent with the NAT Network configuration established in Lab 0.

6.4 Screenshots:



```
(kali@kali)~  
$ ip a  
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000  
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00  
    inet 127.0.0.1/8 scope host lo  
        valid_lft forever preferred_lft forever  
    inet6 ::1/128 scope host noprefixroute  
        valid_lft forever preferred_lft forever  
2: eth0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000  
    link/ether 08:00:27:5a:87:bc brd ff:ff:ff:ff:ff:ff  
    inet 10.0.2.15/24 brd 10.0.2.255 scope global dynamic noprefixroute eth0  
        valid_lft 569sec preferred_lft 569sec  
    inet6 fe80::e9c7:f388:1aaf:5a2/64 scope link noprefixroute  
        valid_lft forever preferred_lft forever  
  
(kali@kali)~  
$
```





6.5 Observation:

eth0 was the only interface showing live traffic activity in Wireshark's sparkline preview, confirming it as the correct capture point for this VM.

7. Task 2: Idle Capture Baseline

7.1: Objective:

Establish a baseline of network activity on eth0 with no deliberate user-generated traffic, to observe whether background traffic occurs without direct action.

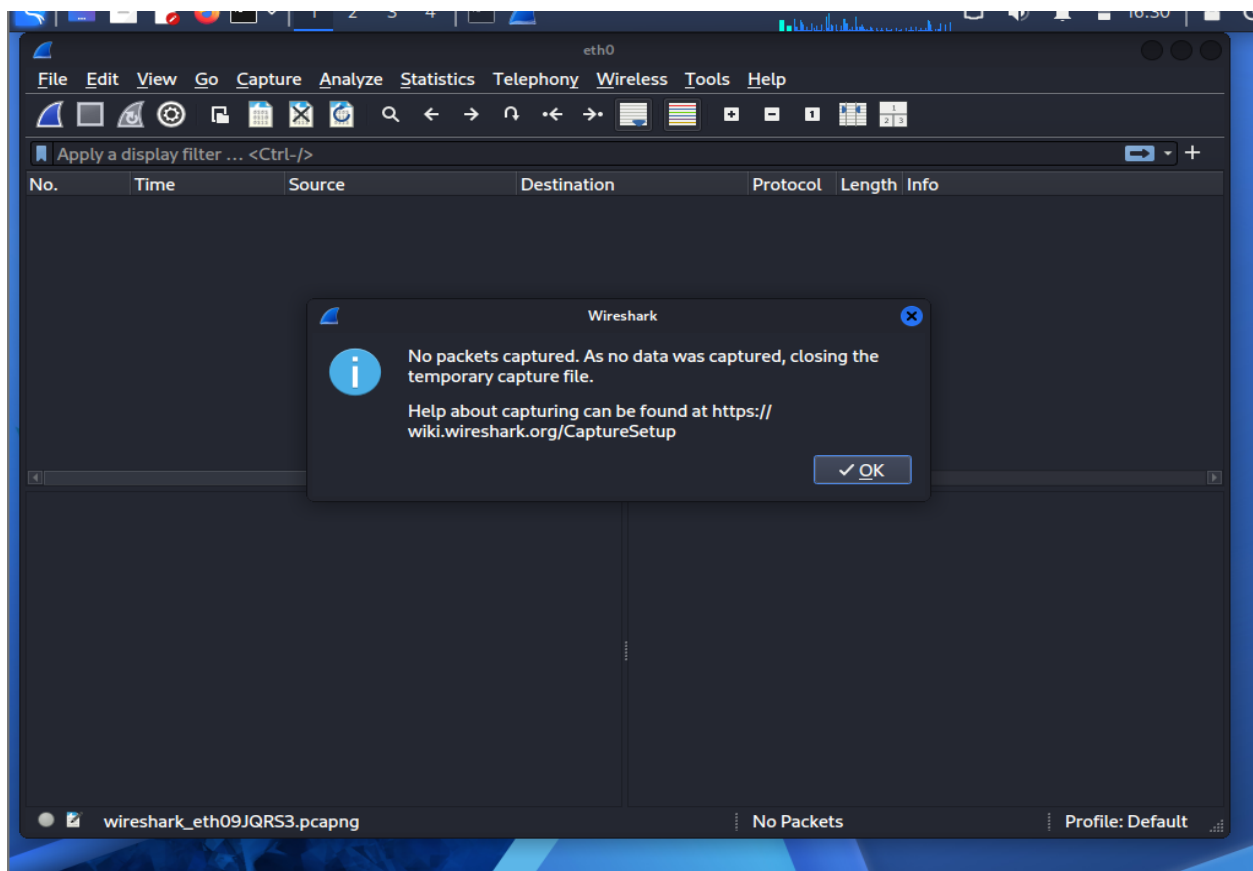
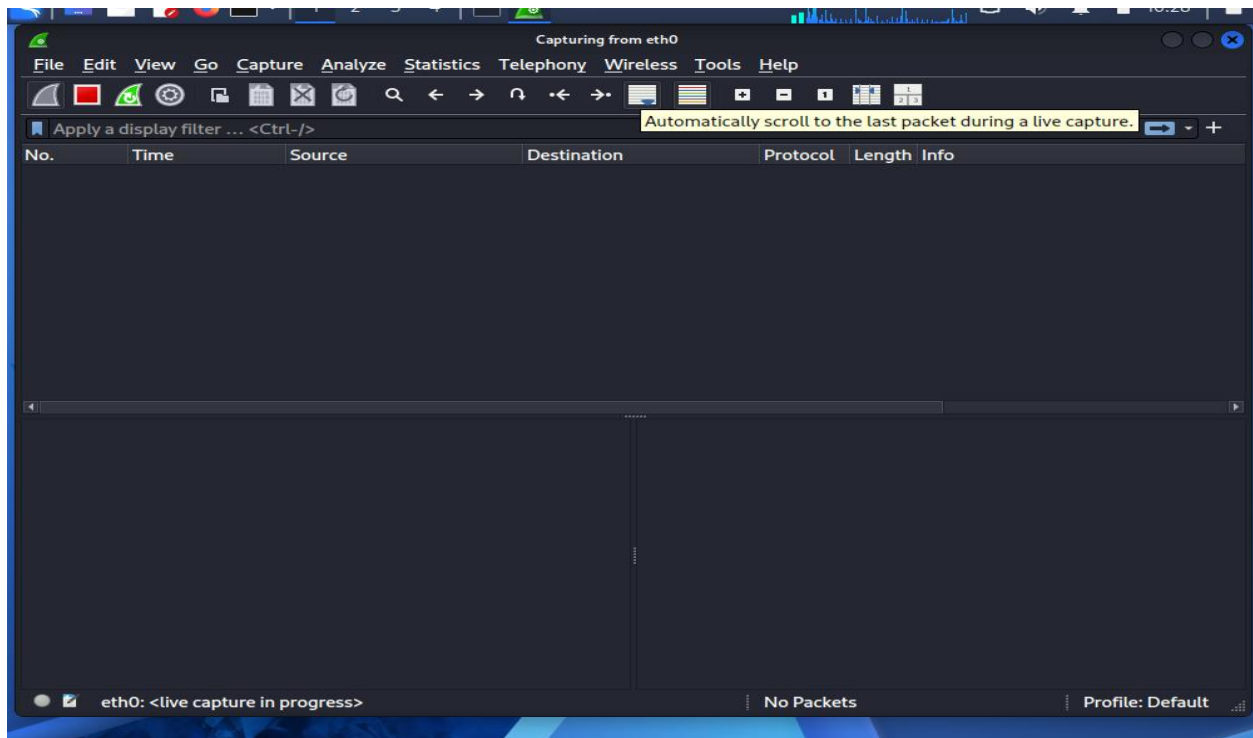
7.2 Procedure:

A live capture was started on eth0 and left running for approximately 30 seconds with no commands executed and no user interaction with the VM.

7.3 Result:

0 packets captured. Wireshark reported "No packets captured. As no data was captured, closing the temporary capture file."

7.4 Screenshots:



7.5 Analysis:

With only one host active on this NAT Network segment, there were no other devices generating broadcast, multicast, or ARP traffic during the capture window, and no DHCP lease renewal happened to fall within this specific 30-second period. This result demonstrates that idle background traffic is not guaranteed on a quiet segment. It depends on factors such as DHCP lease timers, ARP cache expiry, and the presence of other active hosts sharing the network. A busier network segment, or a longer capture window, would likely have produced some background traffic (e.g. the DHCP renewal and ARP exchange observed later, in the background of the Task 6 capture).

7.6 Conclusion:

A quiet result during idle capture is itself a valid and informative finding, not a failed experiment. It establishes that this NAT Network segment, with a single host, produces no traffic without deliberate action, providing a useful baseline against which the traffic generated in later tasks can be compared.

8. Task 3: Traffic Generation (ICMP + DNS)

8.1: Objective:

Generate real network traffic using a common diagnostic tool (ping) and observe the full set of protocols involved in resolving and reaching a remote host.

8.2: Procedure:

A live capture was started on eth0, followed by:

```
ping google.com -c 4
```

The capture was stopped once the command completed, then saved as lab01-task3-traffic-generation.pcapng and exported to text using:

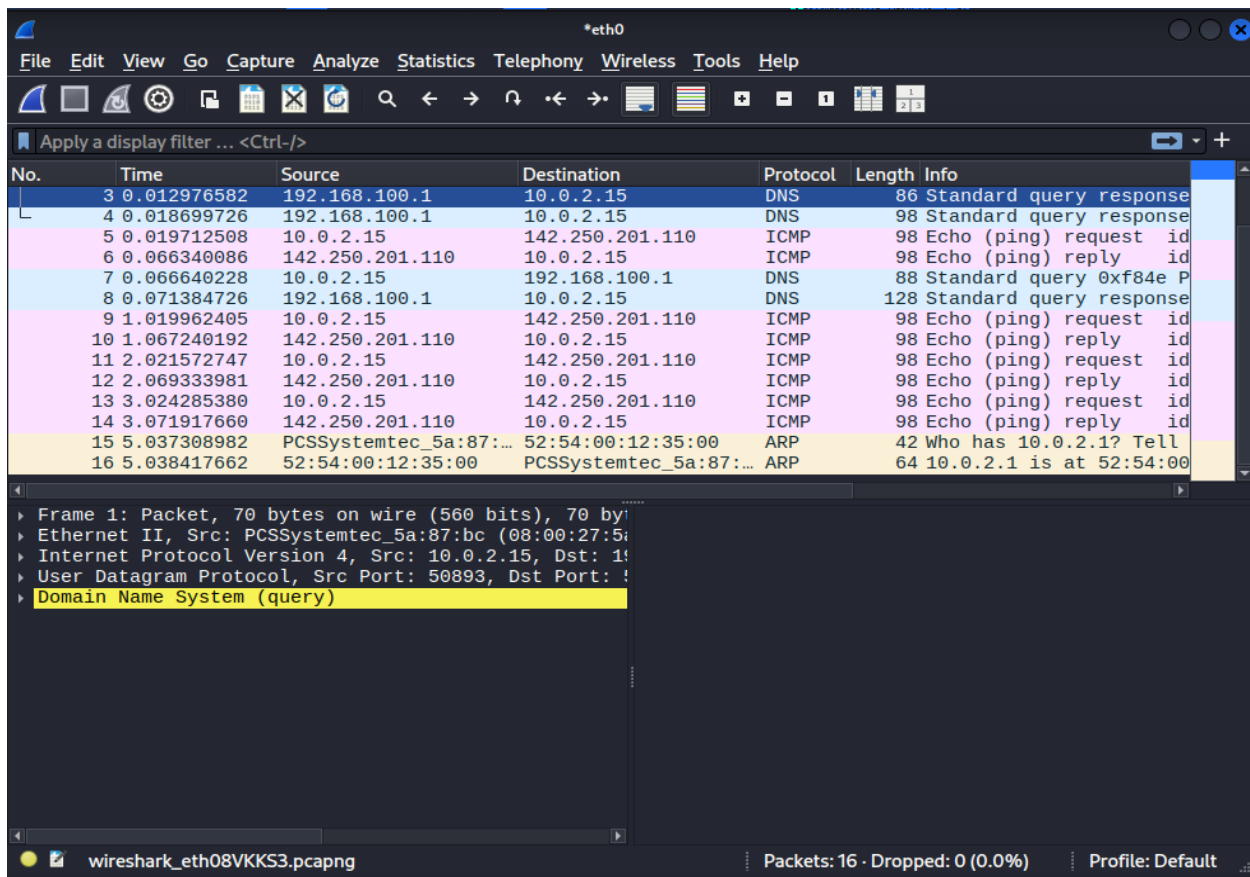
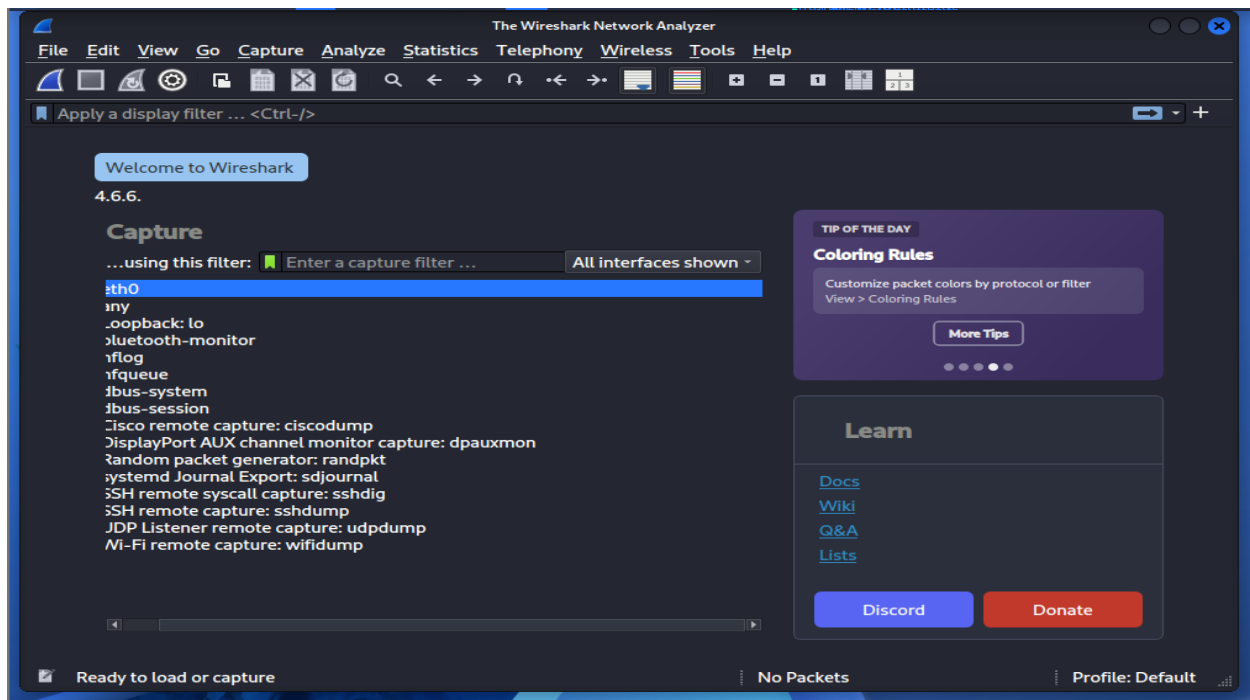
```
tshark -r lab01-task3-traffic-generation.pcapng > lab01-task3-traffic-generation.txt
```

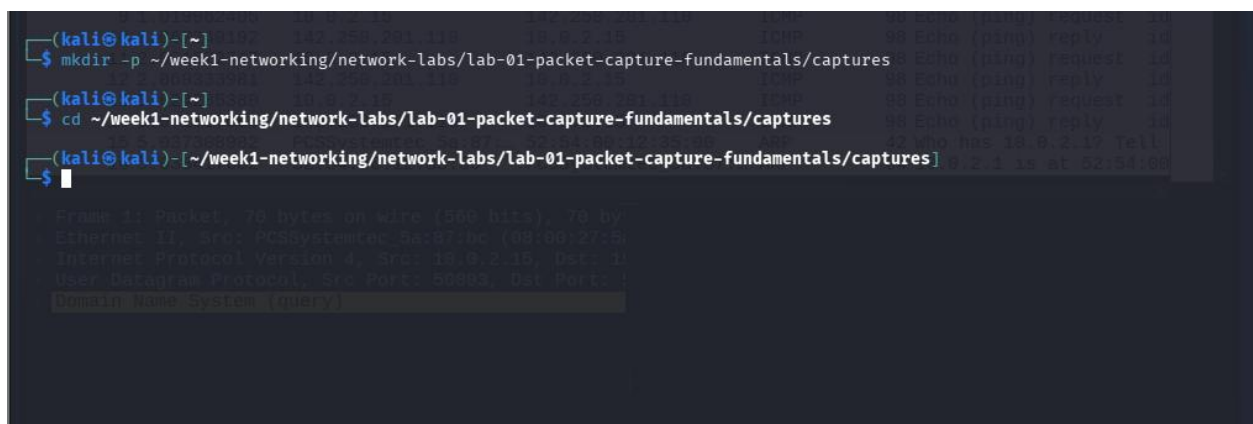
8.3: Result:

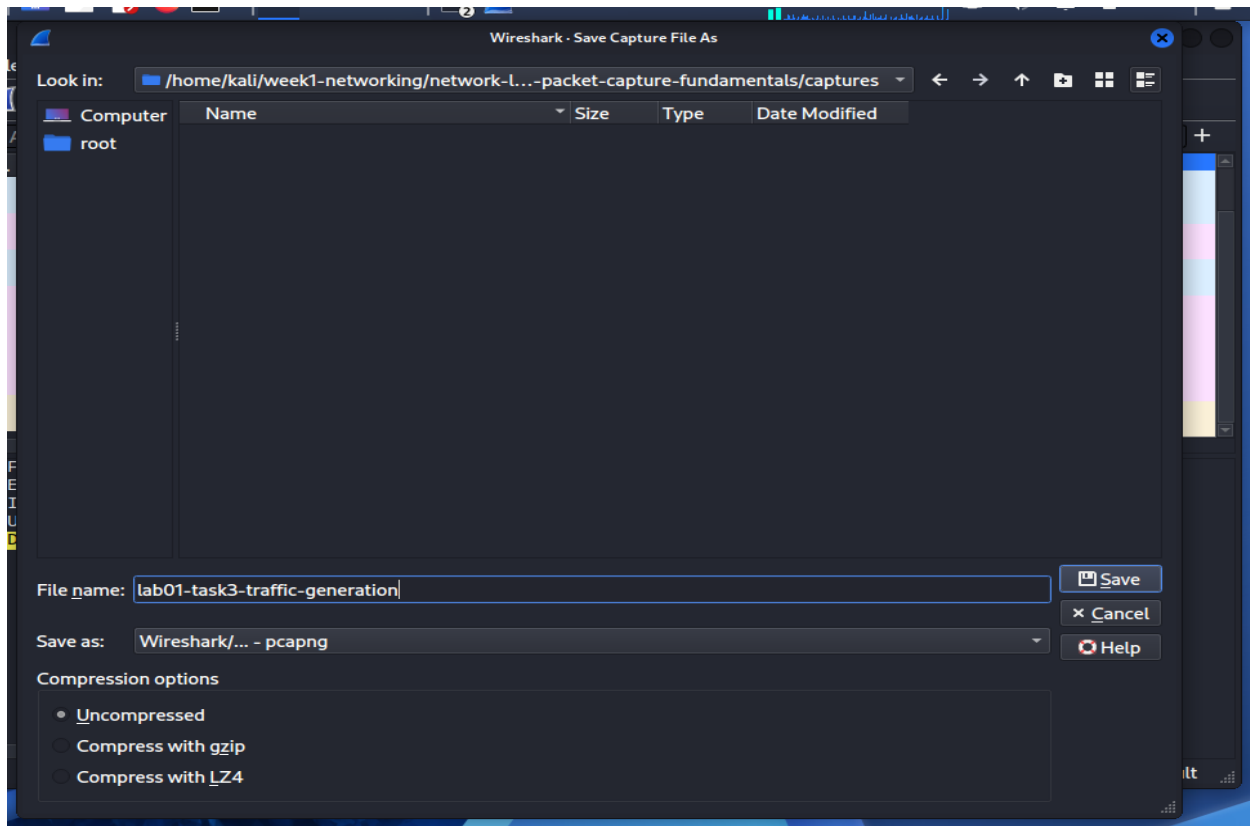
16 packets captured, spanning three distinct protocols:

Packets	Protocol	Description
1–2, 4	DNS	A and AAAA queries for google.com
3	DNS	A record response: google.com → 142.250.201.110
5–6, 9–14	ICMP	4× Echo Request/Reply pairs to 142.250.201.110
7–8	DNS	Reverse PTR lookup for the resolved IP
15–16	ARP	Gateway MAC resolution (Who has 10.0.2.1?)

8.4: Screenshots:







```
cd /home/kali/week1-networking/network-labs/lab-01-packet-capture-fundamentals/captures
(kali@kali)-[~/week1-networking/network-labs/lab-01-packet-capture-fundamentals/captures]
$ tshark -r lab01-task3-traffic-generation.pcapng > lab01-task3-traffic-generation.txt

(kali@kali)-[~/week1-networking/network-labs/lab-01-packet-capture-fundamentals/captures]
$ ls
lab01-task3-traffic-generation.pcapng  lab01-task3-traffic-generation.txt

(kali@kali)-[~/week1-networking/network-labs/lab-01-packet-capture-fundamentals/captures]
$ cat lab01-task3-traffic-generation.txt
1 0.000000000 10.0.2.15 → 192.168.100.1 DNS 70 Standard query 0x6066 A google.com
2 0.000520358 10.0.2.15 → 192.168.100.1 DNS 70 Standard query 0x5c62 AAAA google.com
3 0.012976582 192.168.100.1 → 10.0.2.15 DNS 86 Standard query response 0x6066 A google.com A 142.250.201.110
4 0.018699726 192.168.100.1 → 10.0.2.15 DNS 98 Standard query response 0x5c62 AAAA google.com AAAA 2a00:1450:4019:80f:
:200e
5 0.019712508 10.0.2.15 → 142.250.201.110 ICMP 98 Echo (ping) request id=0x2065, seq=1/256, ttl=64
6 0.066340086 142.250.201.110 → 10.0.2.15 ICMP 98 Echo (ping) reply id=0x2065, seq=1/256, ttl=64 (request in 5)
7 0.066640228 10.0.2.15 → 192.168.100.1 DNS 88 Standard query 0xf84e PTR 110.201.250.142.in-addr.arpa
8 0.071384726 192.168.100.1 → 10.0.2.15 DNS 128 Standard query response 0xf84e PTR 110.201.250.142.in-addr.arpa PTR pn
fjra-ai-in-f14.1e100.net
9 1.019962405 10.0.2.15 → 142.250.201.110 ICMP 98 Echo (ping) request id=0x2065, seq=2/512, ttl=64
10 1.067240192 142.250.201.110 → 10.0.2.15 ICMP 98 Echo (ping) reply id=0x2065, seq=2/512, ttl=64 (request in 9)
11 2.021572747 10.0.2.15 → 142.250.201.110 ICMP 98 Echo (ping) request id=0x2065, seq=3/768, ttl=64
12 2.069333981 142.250.201.110 → 10.0.2.15 ICMP 98 Echo (ping) reply id=0x2065, seq=3/768, ttl=64 (request in 11)
13 3.024285380 10.0.2.15 → 142.250.201.110 ICMP 98 Echo (ping) request id=0x2065, seq=4/1024, ttl=64
14 3.071917660 142.250.201.110 → 10.0.2.15 ICMP 98 Echo (ping) reply id=0x2065, seq=4/1024, ttl=64 (request in 13)
15 5.037308982 PCSSystemtec_5a:87:bc → 52:54:00:12:35:00 ARP 42 Who has 10.0.2.1? Tell 10.0.2.15
16 5.038417662 52:54:00:12:35:00 → PCSSystemtec_5a:87:bc ARP 64 10.0.2.1 is at 52:54:00:12:35:00

(kali@kali)-[~/week1-networking/network-labs/lab-01-packet-capture-fundamentals/captures]
$
```


8.5: Analysis:

A single ping command triggered far more network activity than the ICMP request/reply pair alone. Before any ICMP traffic could be sent, the VM first had to resolve the gateway's MAC address via ARP, since Layer 2 delivery requires a MAC address regardless of the destination's IP. It also had to resolve google.com to an IP address via DNS before ICMP could target anything, since ping requires a concrete destination IP. Both AAAA (IPv6) and PTR (reverse) DNS lookups occurred alongside the expected A record query, detail that was only clearly visible in the tshark text export, not immediately obvious from the Wireshark GUI packet list at a glance.

8.6: Conclusion:

This task demonstrates that even simple, single-command network activity generates layered, multi-protocol traffic behind the scenes. Understanding this layered dependency (ARP → DNS → ICMP) is foundational for later packet analysis, since real investigative work requires recognizing which protocol exchanges are prerequisite "setup" traffic versus the actual traffic of interest.

9. Task 4: ICMP Filtering

9.1: Objective:

Apply a display filter to isolate ICMP traffic from the Task 3 capture and verify Echo Request/Reply behavior.

9.2: Procedure:

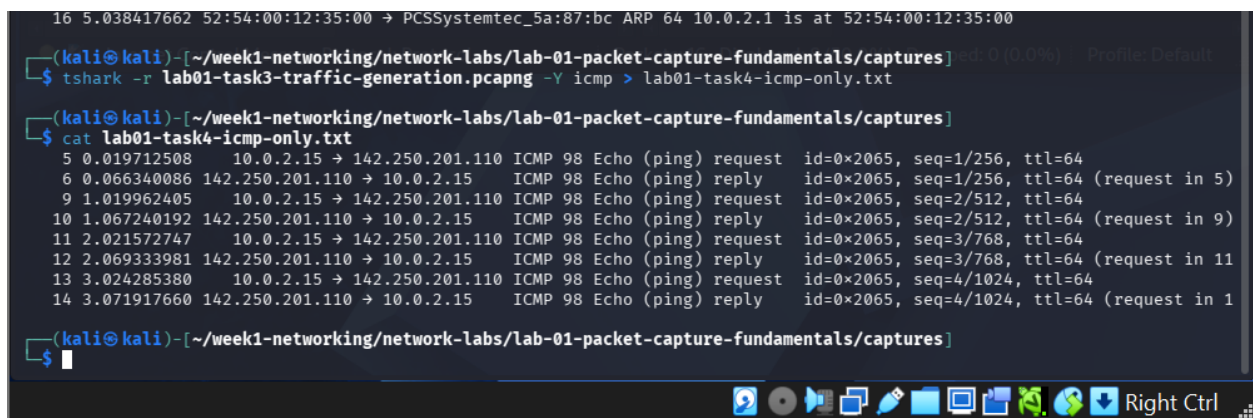
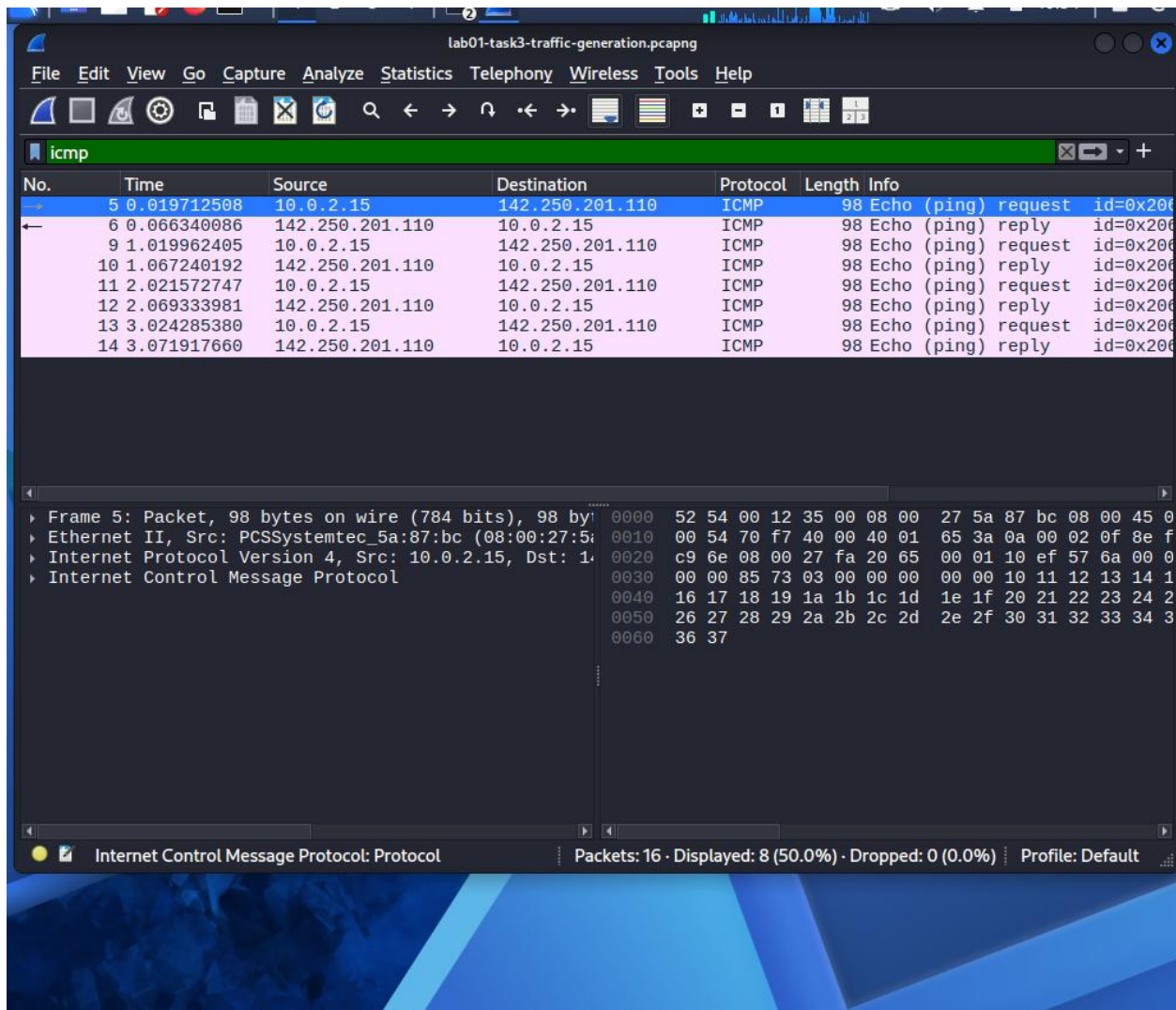
The Task 3 capture (lab01-task3-traffic-generation.pcapng) was filtered using the display filter icmp in Wireshark, and separately exported using:

```
tshark -r lab01-task3-traffic-generation.pcapng -Y icmp > lab01-task4-icmp-only.txt
```

9.3: Result:

4 Echo Requests and 4 Echo Replies, a clean 1:1 pairing matching ping -c 4 with 0% packet loss. All packets shared the same ICMP identifier (id=0x2065), with sequence numbers incrementing from 1 to 4, confirming they belonged to a single ping session.

9.4: Screenshots:



9.5: Analysis:

Filtering did not alter or discard the underlying capture. The full 16-packet file remained intact, with the icmp filter simply narrowing the *view* to 8 of the 16 packets. This distinction, introduced in Section 3.3, is directly demonstrated here: the same .pcapng file was queried twice (once via GUI filter, once via tshark -Y) and produced identical, consistent results.

9.6: Conclusion:

Display filters are a fast, non-destructive way to isolate a protocol of interest within a larger capture, making them essential for efficiently locating relevant evidence in real investigative scenarios where captures may contain thousands of packets.

10. Task 5: DNS Resolution

10.1: Objective:

Capture and analyze the DNS query/response exchange generated by a name resolution tool.

10.2: Procedure:

A live capture was started on eth0, followed by:

```
dig google.com
```

The capture was stopped, filtered using dns, saved as lab01-task5-dns.pcapng, and exported via:

```
tshark -r lab01-task5-dns.pcapng > lab01-task5-dns.txt
```

10.3: Result:

4 packets captured:

Packet	Protocol	Description
1	DNS	Standard query: A record for google.com
2	DNS	Standard query response: google.com → 142.250.200.174
3-4	ARP	Gateway MAC resolution

10.4: Screenshots:

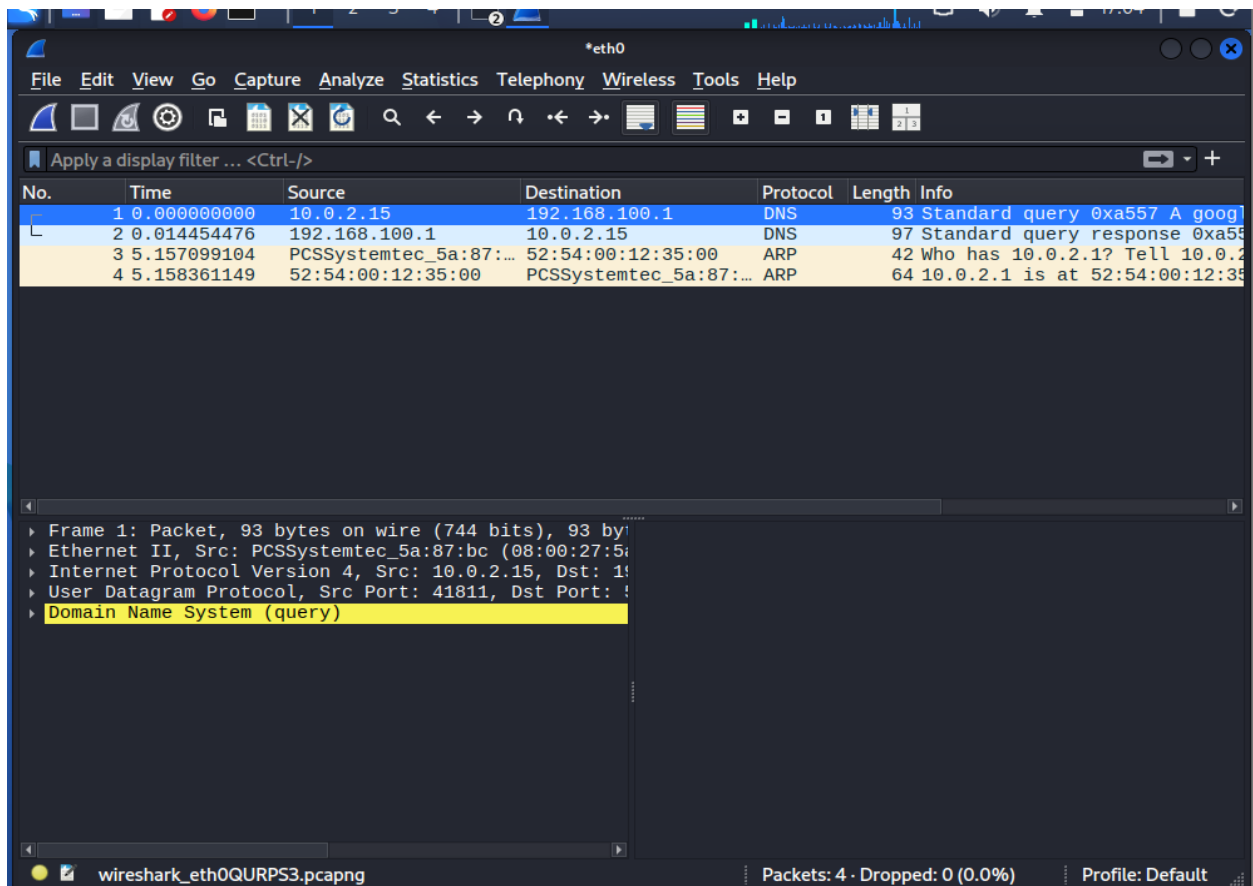
```
(kali@kali)-[~]
$ dig google.com

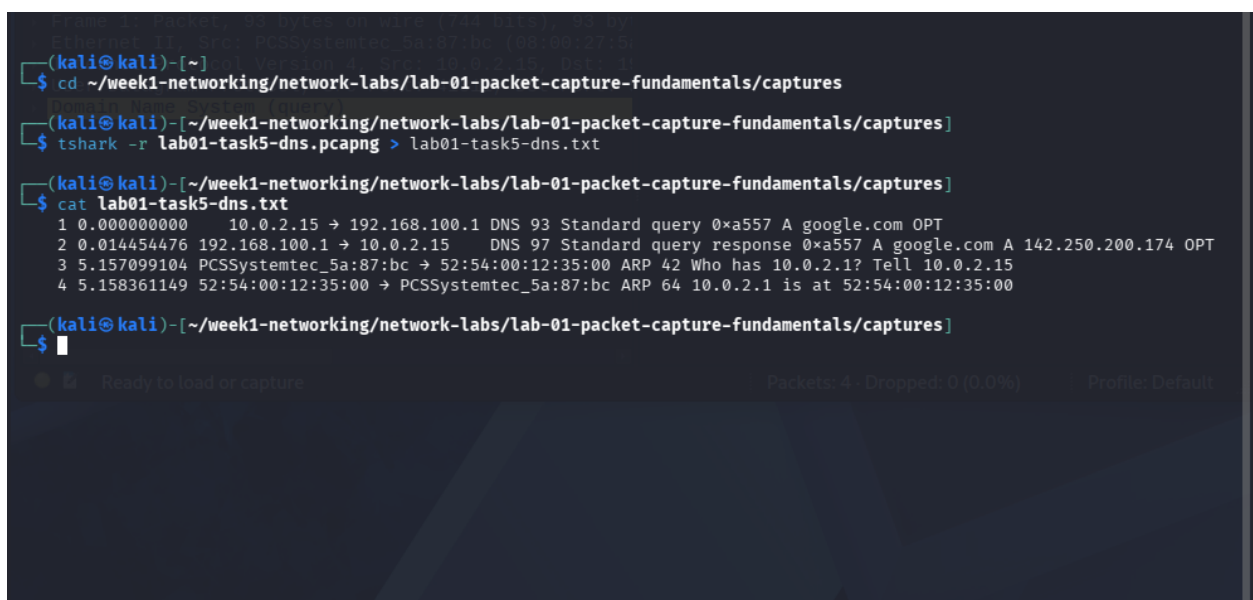
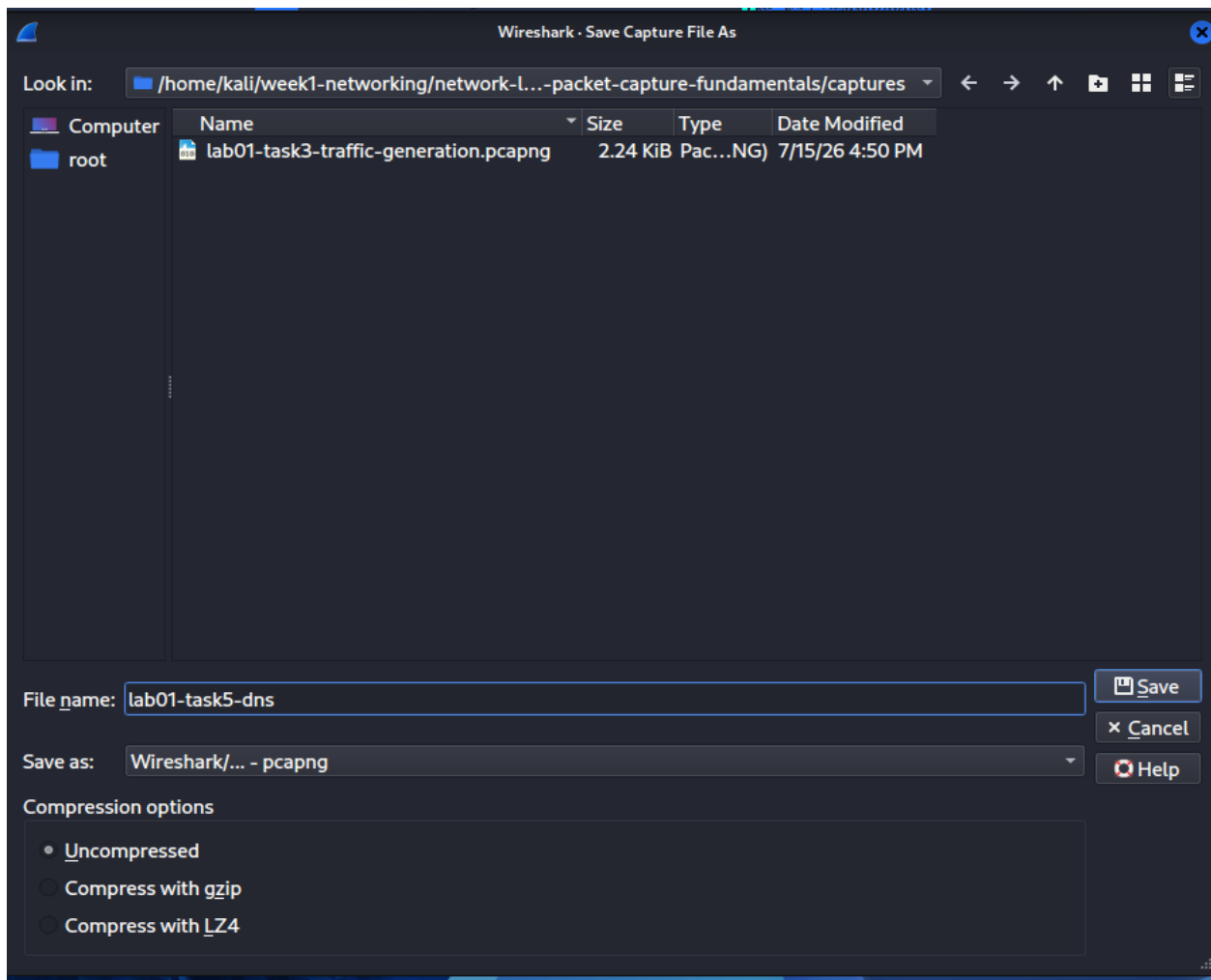
; <<>> DiG 9.20.23-1-Debian <<>> google.com
;; global options: +cmd
;; Got answer:
;; ->HEADER<- opcode: QUERY, status: NOERROR, id: 42327
;; flags: qr rd ra; QUERY: 1, ANSWER: 1, AUTHORITY: 0, ADDITIONAL: 1

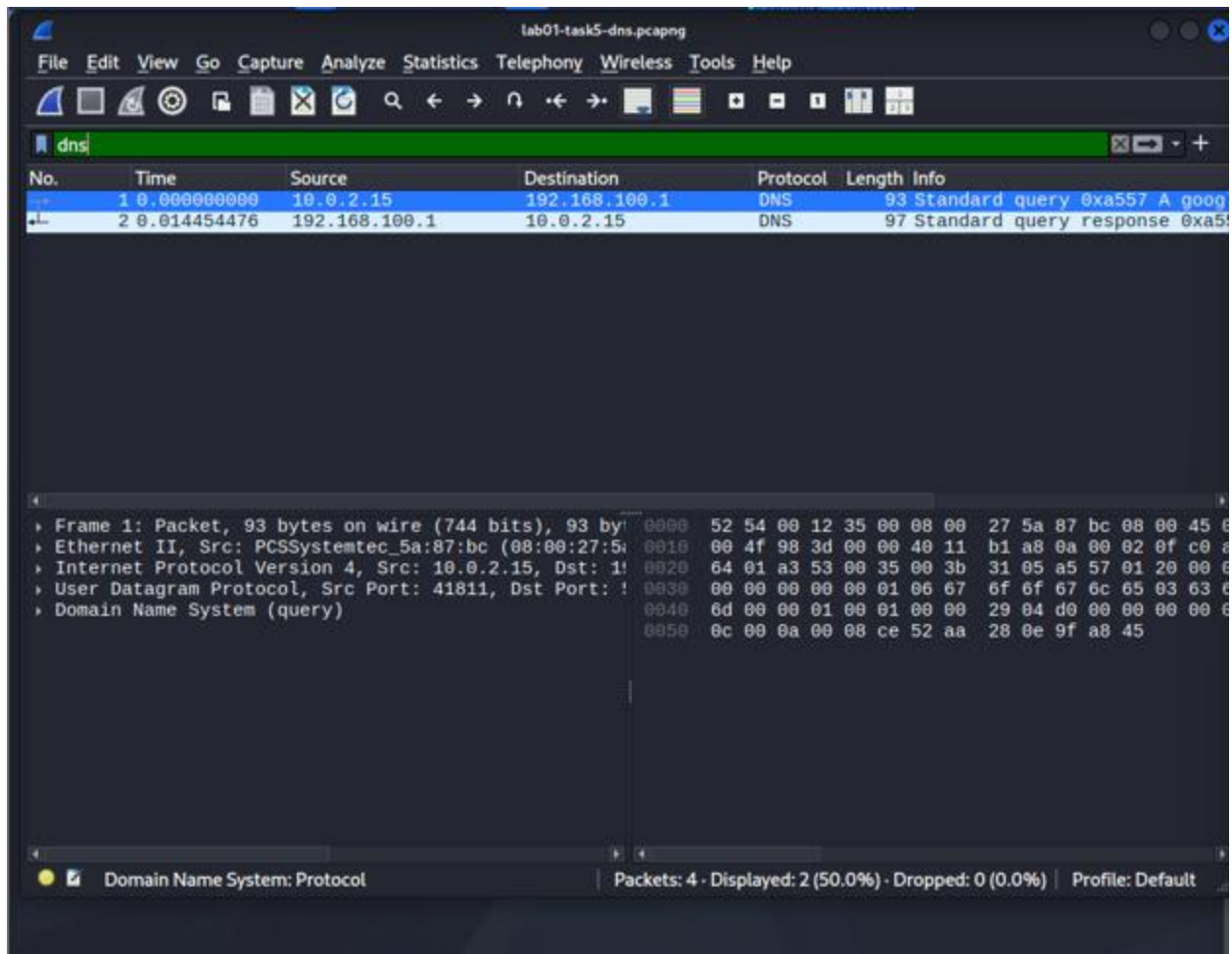
;; OPT PSEUDOSECTION:
; EDNS: version: 0, flags:; udp: 1232
;; QUESTION SECTION:
;google.com.                IN      A

;; ANSWER SECTION:
google.com.                123     IN      A      142.250.200.174

;; Query time: 16 msec
;; SERVER: 192.168.100.1#53(192.168.100.1) (UDP)
;; WHEN: Wed Jul 15 17:03:04 EDT 2026
;; MSG SIZE rcvd: 55
Frame 1: Packet, 93 bytes on wire (744 bits), 93 by
Ethernet II, Src: PCSSystemtec_5a:87:bc (08:00:27:5
(kali@kali)-[~]
$ tcpdump -i eth0 -s 0 -A 'udp port 53 or icmp'
tcpdump: listening on eth0, link-type EN10MB (Ethernet), capture size 0 bytes
0.000000000 192.168.100.1 > 10.0.2.15: 53 [Standard query] Seq=42327
0.014454476 192.168.100.1 > 10.0.2.15: 53 [Standard query response] Seq=42327
5.157099104 PCSSystemtec_5a:87:bc > 52:54:00:12:35:00: 42 [Who has 10.0.2.1? Tell 10.0.2.1]
5.158361149 52:54:00:12:35:00 > PCSSystemtec_5a:87:bc: 64 [10.0.2.1 is at 52:54:00:12:35:00]
```







10.5: Analysis:

The query was answered by 192.168.100.1, the DNS server automatically provided by VirtualBox's NAT Network. The exchange used UDP, consistent with standard DNS behavior. UDP is preferred for its low overhead on small, single-round-trip queries, with TCP used only as a fallback when a response exceeds a single UDP datagram's size limit.

10.6: Conclusion:

This task isolates DNS as a distinct, minimal protocol exchange, reinforcing the dependency observed in Task 3: nearly every outbound connection to a named host begins with a DNS lookup before any application-layer traffic can occur.

11. Task 6: HTTPS/TLS Capture

11.1: Objective:

Capture and analyze a TCP connection secured with TLS, observing the handshake, encrypted data transfer, and connection teardown.

11.2: Procedure:

A live capture was started on eth0, followed by:

```
curl https://google.com
```

The capture was stopped, saved in full as lab01-task6-http.pcapng, filtered using tcp for inspection, and exported via:

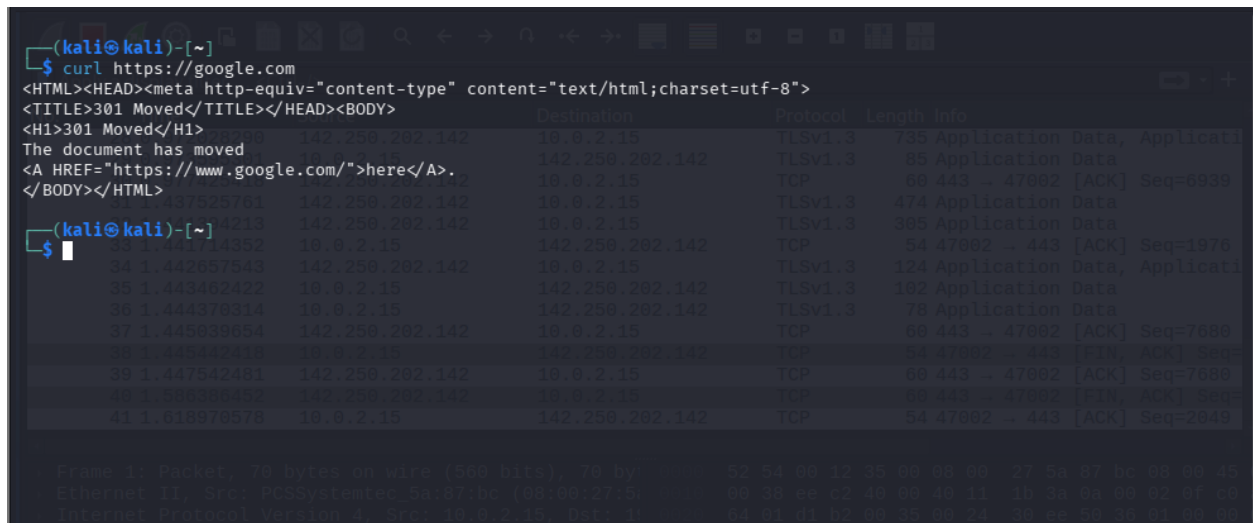
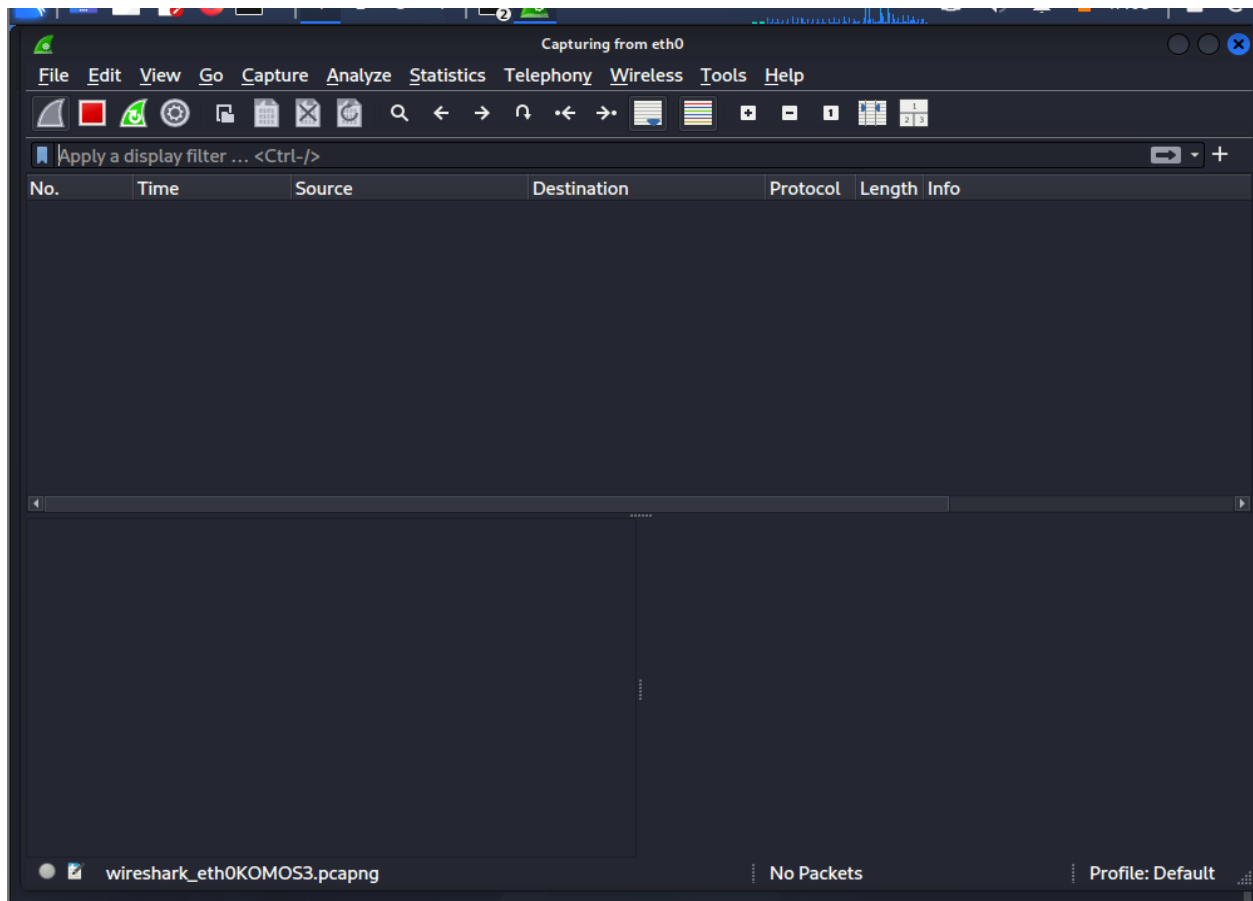
```
tshark -r lab01-task6-http.pcapng > lab01-task6-http.txt
```

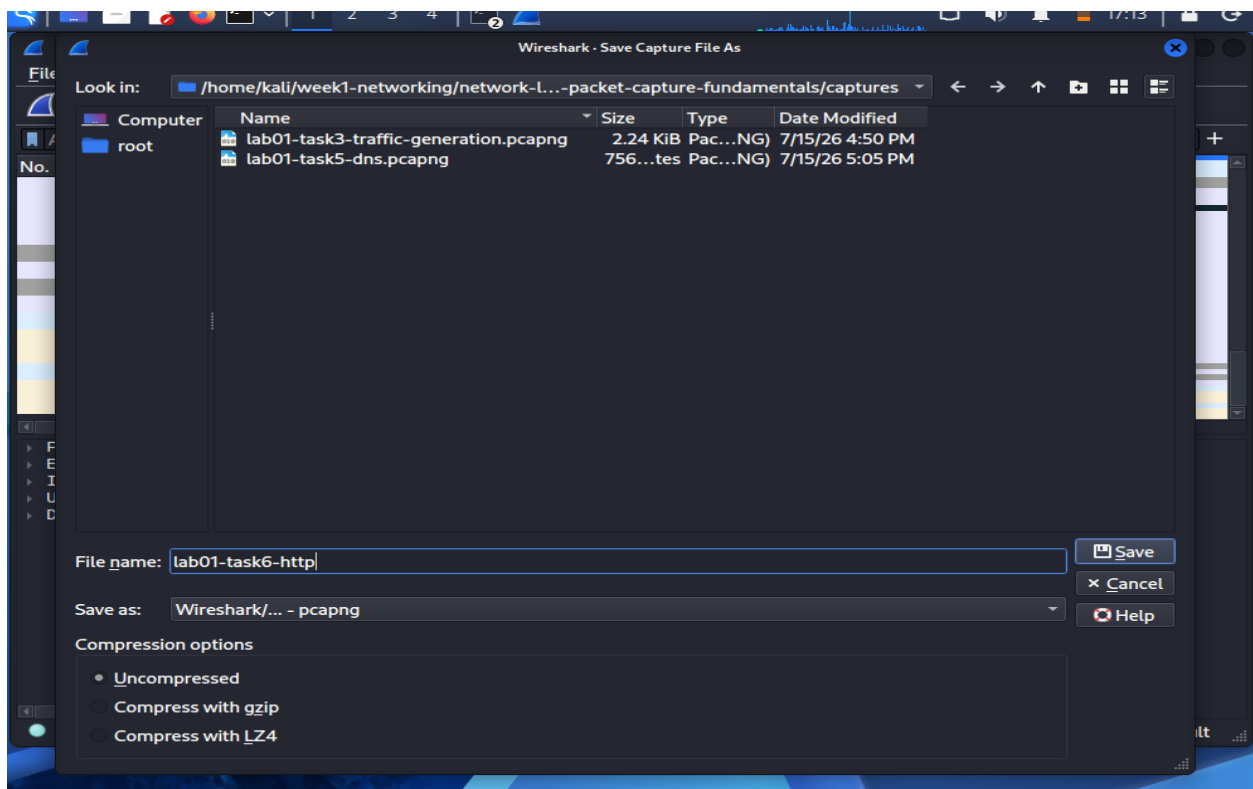
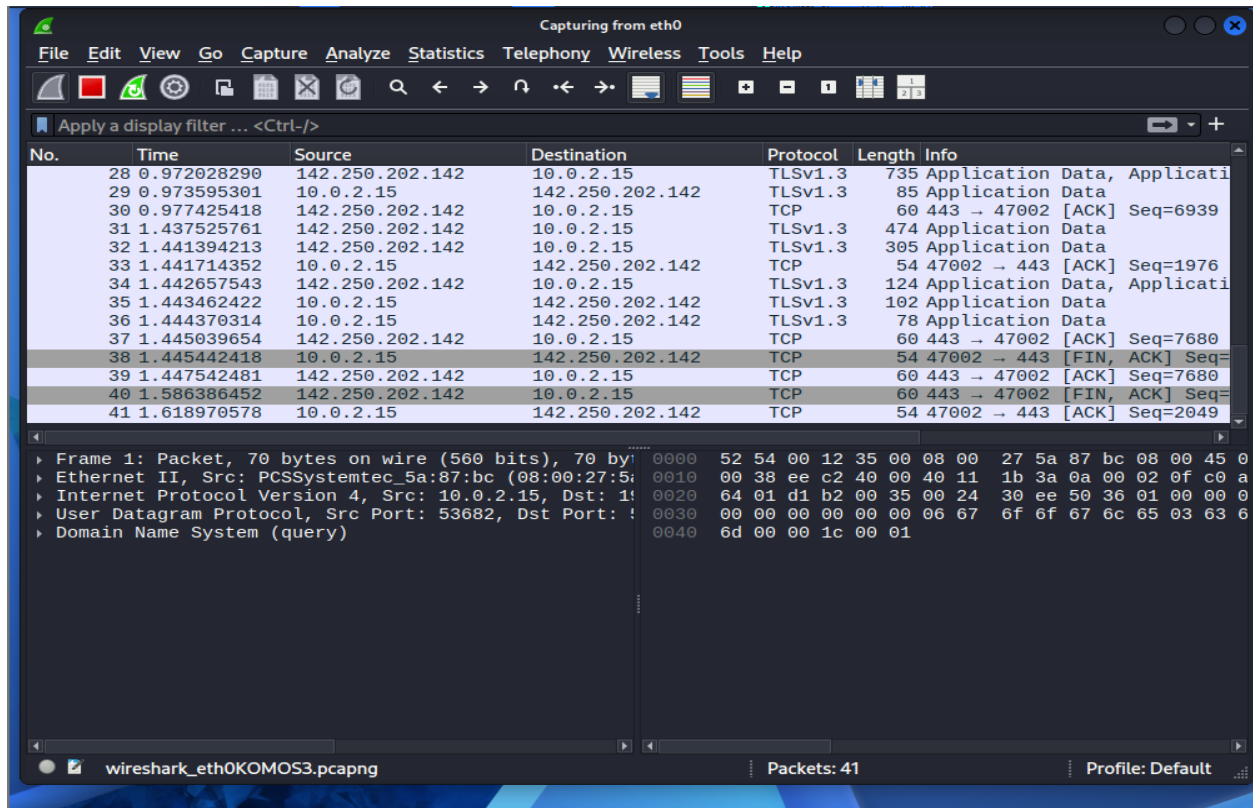
11.3: Result:

47 packets captured (37 matching the tcp filter), showing the following sequence:

Packets	Event
1–4	DNS resolution of google.com
5–7	TCP 3-way handshake (SYN → SYN, ACK → ACK) on port 443
8	TLS Client Hello (SNI=google.com, ALPN=h2/http1.1)
10	TCP retransmission (one unacknowledged segment resent)
12–19	TLS Server Hello, Change Cipher Spec
20–37	Encrypted Application Data (both directions)
38–41	TCP teardown (FIN, ACK exchanged both directions)

11.4: Screenshots:





```

(kali@kali)-[~]
$ cd ~/week1-networking/network-labs/lab-01-packet-capture-fundamentals/captures

(kali@kali)-[~/week1-networking/network-labs/lab-01-packet-capture-fundamentals/captures]
$ tshark -r lab01-task6-http.pcapng > lab01-task6-http.txt

(kali@kali)-[~/week1-networking/network-labs/lab-01-packet-capture-fundamentals/captures]
$ cat lab01-task6-http.txt
1 0.000000000 10.0.2.15 → 192.168.100.1 DNS 70 Standard query 0x5036 AAAA google.com
2 0.000494138 10.0.2.15 → 192.168.100.1 DNS 70 Standard query 0xa328 A google.com
3 0.025065606 192.168.100.1 → 10.0.2.15 DNS 98 Standard query response 0x5036 AAAA google.com AAAA 2a00:1450:4019:
810::200e
4 0.025066234 192.168.100.1 → 10.0.2.15 DNS 86 Standard query response 0xa328 A google.com A 142.250.202.142
5 0.032541471 10.0.2.15 → 142.250.202.142 TCP 74 47002 → 443 [SYN] Seq=0 Win=64240 Len=0 MSS=1460 SACK_PERM TSval=
3370998873 TSecr=0 WS=256
6 0.183753235 142.250.202.142 → 10.0.2.15 TCP 60 443 → 47002 [SYN, ACK] Seq=0 Ack=1 Win=65535 Len=0 MSS=1460
7 0.187799135 10.0.2.15 → 142.250.202.142 TCP 54 47002 → 443 [ACK] Seq=1 Ack=1 Win=64240 Len=0
8 0.197585435 10.0.2.15 → 142.250.202.142 TLSv1.2 1748 Client Hello (SNI=google.com)
9 0.200039637 142.250.202.142 → 10.0.2.15 TCP 60 443 → 47002 [ACK] Seq=1 Ack=1461 Win=65535 Len=0
10 0.663531010 10.0.2.15 → 142.250.202.142 TCP 288 [TCP Retransmission] 47002 → 443 [PSH, ACK] Seq=1461 Ack=1 Win=6
4240 Len=234
11 0.665413382 142.250.202.142 → 10.0.2.15 TCP 60 443 → 47002 [ACK] Seq=1 Ack=1695 Win=65535 Len=0
12 0.869267750 142.250.202.142 → 10.0.2.15 TLSv1.3 1466 Server Hello, Change Cipher Spec
13 0.869462054 10.0.2.15 → 142.250.202.142 TCP 54 47002 → 443 [ACK] Seq=1695 Ack=1413 Win=65535 Len=0
14 0.873930410 142.250.202.142 → 10.0.2.15 TCP 1466 443 → 47002 [PSH, ACK] Seq=1413 Ack=1695 Win=65535 Len=1412
15 0.873931001 142.250.202.142 → 10.0.2.15 TCP 1494 443 → 47002 [ACK] Seq=2825 Ack=1695 Win=65535 Len=1440
16 0.873931072 142.250.202.142 → 10.0.2.15 TCP 1494 443 → 47002 [ACK] Seq=4265 Ack=1695 Win=65535 Len=1440
17 0.874017362 10.0.2.15 → 142.250.202.142 TCP 54 47002 → 443 [ACK] Seq=1695 Ack=2825 Win=65535 Len=0
18 0.874088688 10.0.2.15 → 142.250.202.142 TCP 54 47002 → 443 [ACK] Seq=1695 Ack=4265 Win=65535 Len=0
19 0.874369334 10.0.2.15 → 142.250.202.142 TCP 54 47002 → 443 [ACK] Seq=1695 Ack=5705 Win=65535 Len=0
20 0.875409872 142.250.202.142 → 10.0.2.15 TLSv1.3 607 Application Data
21 0.875890141 10.0.2.15 → 142.250.202.142 TCP 54 47002 → 443 [ACK] Seq=1695 Ack=6258 Win=65535 Len=0
22 0.914514505 10.0.2.15 → 142.250.202.142 TLSv1.3 134 Change Cipher Spec, Application Data
23 0.915332343 142.250.202.142 → 10.0.2.15 TCP 60 443 → 47002 [ACK] Seq=6258 Ack=1775 Win=65535 Len=0
24 0.916132185 10.0.2.15 → 142.250.202.142 TLSv1.3 176 Application Data
25 0.916869544 142.250.202.142 → 10.0.2.15 TCP 60 443 → 47002 [ACK] Seq=6258 Ack=1897 Win=65535 Len=0
26 0.916886718 10.0.2.15 → 142.250.202.142 TLSv1.3 102 Application Data
27 0.917729569 142.250.202.142 → 10.0.2.15 TCP 60 443 → 47002 [ACK] Seq=6258 Ack=1945 Win=65535 Len=0
28 0.972028290 142.250.202.142 → 10.0.2.15 TLSv1.3 735 Application Data, Application Data, Application Data
29 0.973595301 10.0.2.15 → 142.250.202.142 TLSv1.3 85 Application Data
30 0.977425418 142.250.202.142 → 10.0.2.15 TCP 60 443 → 47002 [ACK] Seq=6939 Ack=1976 Win=65535 Len=0
31 1.437525761 142.250.202.142 → 10.0.2.15 TLSv1.3 474 Application Data
32 1.441394213 142.250.202.142 → 10.0.2.15 TLSv1.3 305 Application Data
33 1.441714352 10.0.2.15 → 142.250.202.142 TCP 54 47002 → 443 [ACK] Seq=1976 Ack=7610 Win=65535 Len=0
34 1.442657543 142.250.202.142 → 10.0.2.15 TLSv1.3 124 Application Data, Application Data
35 1.443462422 10.0.2.15 → 142.250.202.142 TLSv1.3 102 Application Data
36 1.444370314 10.0.2.15 → 142.250.202.142 TLSv1.3 78 Application Data
37 1.445039654 142.250.202.142 → 10.0.2.15 TCP 60 443 → 47002 [ACK] Seq=7680 Ack=2048 Win=65535 Len=0
38 1.445442418 10.0.2.15 → 142.250.202.142 TCP 54 47002 → 443 [FIN, ACK] Seq=2048 Ack=7680 Win=65535 Len=0
39 1.447542481 142.250.202.142 → 10.0.2.15 TCP 60 443 → 47002 [ACK] Seq=7680 Ack=2049 Win=65535 Len=0
40 1.586386452 142.250.202.142 → 10.0.2.15 TCP 60 443 → 47002 [FIN, ACK] Seq=7680 Ack=2049 Win=65535 Len=0
41 1.618970578 10.0.2.15 → 142.250.202.142 TCP 54 47002 → 443 [ACK] Seq=2049 Ack=7681 Win=19424 Len=0

24 0.916132185 10.0.2.15 → 142.250.202.142 TLSv1.3 176 Application Data
25 0.916869544 142.250.202.142 → 10.0.2.15 TCP 60 443 → 47002 [ACK] Seq=6258 Ack=1897 Win=65535 Len=0
26 0.916886718 10.0.2.15 → 142.250.202.142 TLSv1.3 102 Application Data
27 0.917729569 142.250.202.142 → 10.0.2.15 TCP 60 443 → 47002 [ACK] Seq=6258 Ack=1945 Win=65535 Len=0
28 0.972028290 142.250.202.142 → 10.0.2.15 TLSv1.3 735 Application Data, Application Data, Application Data
29 0.973595301 10.0.2.15 → 142.250.202.142 TLSv1.3 85 Application Data
30 0.977425418 142.250.202.142 → 10.0.2.15 TCP 60 443 → 47002 [ACK] Seq=6939 Ack=1976 Win=65535 Len=0
31 1.437525761 142.250.202.142 → 10.0.2.15 TLSv1.3 474 Application Data
32 1.441394213 142.250.202.142 → 10.0.2.15 TLSv1.3 305 Application Data
33 1.441714352 10.0.2.15 → 142.250.202.142 TCP 54 47002 → 443 [ACK] Seq=1976 Ack=7610 Win=65535 Len=0
34 1.442657543 142.250.202.142 → 10.0.2.15 TLSv1.3 124 Application Data, Application Data
35 1.443462422 10.0.2.15 → 142.250.202.142 TLSv1.3 102 Application Data
36 1.444370314 10.0.2.15 → 142.250.202.142 TLSv1.3 78 Application Data
37 1.445039654 142.250.202.142 → 10.0.2.15 TCP 60 443 → 47002 [ACK] Seq=7680 Ack=2048 Win=65535 Len=0
38 1.445442418 10.0.2.15 → 142.250.202.142 TCP 54 47002 → 443 [FIN, ACK] Seq=2048 Ack=7680 Win=65535 Len=0
39 1.447542481 142.250.202.142 → 10.0.2.15 TCP 60 443 → 47002 [ACK] Seq=7680 Ack=2049 Win=65535 Len=0
40 1.586386452 142.250.202.142 → 10.0.2.15 TCP 60 443 → 47002 [FIN, ACK] Seq=7680 Ack=2049 Win=65535 Len=0
41 1.618970578 10.0.2.15 → 142.250.202.142 TCP 54 47002 → 443 [ACK] Seq=2049 Ack=7681 Win=19424 Len=0
42 128.309035468 10.0.2.15 → 10.0.2.2 DHCP 324 DHCP Request - Transaction ID 0x7dfa837b
43 128.347306141 PCSSystemtec_34:73:29 → Broadcast ARP 60 Who has 10.0.2.15? Tell 10.0.2.2
44 128.347345101 PCSSystemtec_5a:87:bc → PCSSystemtec_34:73:29 ARP 42 10.0.2.15 is at 08:00:27:5a:87:bc
45 128.347306701 10.0.2.2 → 10.0.2.15 DHCP 590 DHCP ACK - Transaction ID 0x7dfa837b
46 133.362894356 PCSSystemtec_5a:87:bc → PCSSystemtec_34:73:29 ARP 42 Who has 10.0.2.2? Tell 10.0.2.15
47 133.365897221 PCSSystemtec_34:73:29 → PCSSystemtec_5a:87:bc ARP 60 10.0.2.2 is at 08:00:27:34:73:29

(kali@kali)-[~/week1-networking/network-labs/lab-01-packet-capture-fundamentals/captures]
$

```

The screenshot shows a Wireshark packet capture of an HTTP transaction. The packet list on the left shows 47 packets. The selected packet is packet 5, an HTTP GET request from 10.0.2.15 to 142.250.202.142. The packet details pane shows the request line: GET / HTTP/1.1. The packet bytes pane shows the raw data of the request.

No.	Time	Source	Destination	Protocol	Length	Info
5	0.032541471	10.0.2.15	142.250.202.142	TCP	74	47002 → 443 [SYN] Seq=0 Win=64240 Len=0 MSS=1460 SACK_PERM TSval=3370998873 TSecr=0 WS=256
6	0.033753235	142.250.202.142	10.0.2.15	TCP	60	443 → 47002 [ACK] Seq=1695 Ack=1 Win=65535 Len=0 MSS=1460
7	0.187799135	10.0.2.15	142.250.202.142	TCP	54	47002 → 443 [ACK] Seq=1 Ack=1 Win=64240 Len=0
8	0.197585435	10.0.2.15	142.250.202.142	TLSv1.3	1748	Client Hello [SNI=google.com]
9	0.200030937	142.250.202.142	10.0.2.15	TCP	60	443 → 47002 [ACK] Seq=1 Ack=1 Win=65535 Len=0
10	0.003310110	10.0.2.15	142.250.202.142	TCP	208	[TCP Reset] Seq=1461 Ack=1 Win=64240 Len=234
11	0.665413382	142.250.202.142	10.0.2.15	TCP	60	443 → 47002 [ACK] Seq=1 Ack=1695 Win=65535 Len=0
12	0.669207750	142.250.202.142	10.0.2.15	TLSv1.3	1460	Server Hello, Change Cipher Spec
13	0.669462954	10.0.2.15	142.250.202.142	TCP	54	47002 → 443 [ACK] Seq=1695 Ack=1413 Win=65535 Len=0
14	0.673930419	142.250.202.142	10.0.2.15	TCP	1466	443 → 47002 [PSH, ACK] Seq=1413 Ack=1695 Win=65535 Len=1412 [TCP PDU reassembled in 20]
15	0.673931901	142.250.202.142	10.0.2.15	TCP	1494	443 → 47002 [ACK] Seq=2025 Ack=1695 Win=65535 Len=1440 [TCP PDU reassembled in 20]
16	0.673931972	142.250.202.142	10.0.2.15	TCP	1494	443 → 47002 [ACK] Seq=4265 Ack=1695 Win=65535 Len=1440 [TCP PDU reassembled in 20]
17	0.674017362	10.0.2.15	142.250.202.142	TCP	54	47002 → 443 [ACK] Seq=1695 Ack=2825 Win=65535 Len=0
18	0.674086868	10.0.2.15	142.250.202.142	TCP	54	47002 → 443 [ACK] Seq=1695 Ack=4265 Win=65535 Len=0
19	0.674369334	10.0.2.15	142.250.202.142	TCP	54	47002 → 443 [ACK] Seq=1695 Ack=5705 Win=65535 Len=0
20	0.675469872	142.250.202.142	10.0.2.15	TLSv1.3	607	Application Data
21	0.675890141	10.0.2.15	142.250.202.142	TCP	54	47002 → 443 [ACK] Seq=1695 Ack=6258 Win=65535 Len=0
22	0.914514595	10.0.2.15	142.250.202.142	TLSv1.3	134	Change Cipher Spec, Application Data
23	0.915332343	142.250.202.142	10.0.2.15	TCP	60	443 → 47002 [ACK] Seq=6258 Ack=1775 Win=65535 Len=0
24	0.916132185	10.0.2.15	142.250.202.142	TLSv1.3	176	Application Data
25	0.916869544	142.250.202.142	10.0.2.15	TCP	60	443 → 47002 [ACK] Seq=6258 Ack=1897 Win=65535 Len=0
26	0.916869718	10.0.2.15	142.250.202.142	TLSv1.3	102	Application Data
27	0.917729569	142.250.202.142	10.0.2.15	TCP	60	443 → 47002 [ACK] Seq=6258 Ack=1945 Win=65535 Len=0

Frame 5: Packet, 74 bytes on wire (592 bits), 74 bytes captured (592 bits) on interface eth0, id 0
Ethernet II, Src: PCSysntec-Sa:87:bc (08:00:27:5a:87:bc), Dst: 52:54:00:12:35:00 (52:54:00:12:35:00)
Internet Protocol Version 4, Src: 10.0.2.15, Dst: 142.250.202.142
Transmission Control Protocol, Src Port: 47002, Dst Port: 443, Seq: 0, Len: 0

The screenshot shows a Wireshark packet capture of an HTTP transaction. The packet list on the left shows 47 packets. The selected packet is packet 5, an HTTP GET request from 10.0.2.15 to 142.250.202.142. The packet details pane shows the request line: GET / HTTP/1.1. The packet bytes pane shows the raw data of the request.

No.	Time	Source	Destination	Protocol	Length	Info
20	0.675469872	142.250.202.142	10.0.2.15	TLSv1.3	607	Application Data
21	0.675890141	10.0.2.15	142.250.202.142	TCP	54	47002 → 443 [ACK] Seq=1695 Ack=6258 Win=65535 Len=0
22	0.914514595	10.0.2.15	142.250.202.142	TLSv1.3	134	Change Cipher Spec, Application Data
23	0.915332343	142.250.202.142	10.0.2.15	TCP	60	443 → 47002 [ACK] Seq=6258 Ack=1775 Win=65535 Len=0
24	0.916132185	10.0.2.15	142.250.202.142	TLSv1.3	176	Application Data
25	0.916869544	142.250.202.142	10.0.2.15	TCP	60	443 → 47002 [ACK] Seq=6258 Ack=1897 Win=65535 Len=0
26	0.916869718	10.0.2.15	142.250.202.142	TLSv1.3	102	Application Data
27	0.917729569	142.250.202.142	10.0.2.15	TCP	60	443 → 47002 [ACK] Seq=6258 Ack=1945 Win=65535 Len=0
28	0.972028290	142.250.202.142	10.0.2.15	TLSv1.3	735	Application Data, Application Data, Application Data
29	0.973595301	10.0.2.15	142.250.202.142	TLSv1.3	85	Application Data
30	0.977425410	142.250.202.142	10.0.2.15	TCP	60	443 → 47002 [ACK] Seq=6939 Ack=1976 Win=65535 Len=0
31	1.437525761	142.250.202.142	10.0.2.15	TLSv1.3	474	Application Data
32	1.441394213	142.250.202.142	10.0.2.15	TLSv1.3	395	Application Data
33	1.441714352	10.0.2.15	142.250.202.142	TCP	54	47002 → 443 [ACK] Seq=1976 Ack=7610 Win=65535 Len=0
34	1.442657543	142.250.202.142	10.0.2.15	TLSv1.3	124	Application Data, Application Data
35	1.443462422	10.0.2.15	142.250.202.142	TLSv1.3	102	Application Data
36	1.444370314	10.0.2.15	142.250.202.142	TLSv1.3	78	Application Data
37	1.445039654	142.250.202.142	10.0.2.15	TCP	60	443 → 47002 [ACK] Seq=7680 Ack=2049 Win=65535 Len=0
38	1.445442418	10.0.2.15	142.250.202.142	TCP	54	47002 → 443 [FIN, ACK] Seq=2648 Ack=7680 Win=65535 Len=0
39	1.447542481	142.250.202.142	10.0.2.15	TCP	60	443 → 47002 [ACK] Seq=7680 Ack=2049 Win=65535 Len=0
40	1.448036652	142.250.202.142	10.0.2.15	TCP	60	443 → 47002 [FIN, ACK] Seq=7680 Ack=2049 Win=65535 Len=0
41	6.189705776	10.0.2.15	142.250.202.142	TCP	54	47002 → 443 [ACK] Seq=2049 Ack=7681 Win=19424 Len=0

Frame 5: Packet, 74 bytes on wire (592 bits), 74 bytes captured (592 bits) on interface eth0, id 0
Ethernet II, Src: PCSysntec-Sa:87:bc (08:00:27:5a:87:bc), Dst: 52:54:00:12:35:00 (52:54:00:12:35:00)
Internet Protocol Version 4, Src: 10.0.2.15, Dst: 142.250.202.142
Transmission Control Protocol, Src Port: 47002, Dst Port: 443, Seq: 0, Len: 0

11.5: Analysis:

The connection followed the full TCP/TLS lifecycle: handshake, encryption negotiation, encrypted transfer, and clean teardown. Because HTTPS encrypts application data, the actual HTTP request and response content was not visible in Wireshark, packets 20–37 appear only as opaque "Application Data." This is expected behavior and demonstrates HTTPS functioning correctly: content is protected from on-path inspection, including by an analyst without the relevant private key. A single retransmission (packet 10) was also observed, illustrating that minor network-level packet loss and recovery is normal even in a successful connection.

11.6 Conclusion:

This task illustrates the practical difference between plain text and encrypted traffic at the packet-level, while the connection metadata (IPs, ports, timing, TLS handshake fields) remains visible, the actual content is protected, which is the foundational guarantee HTTPS is designed to provide.

12. Task 7: Packet Inspection**12.1: Objective:**

Perform a detailed, layer-by-layer inspection of a single packet to identify key addressing and protocol fields.

12.2: Procedure:

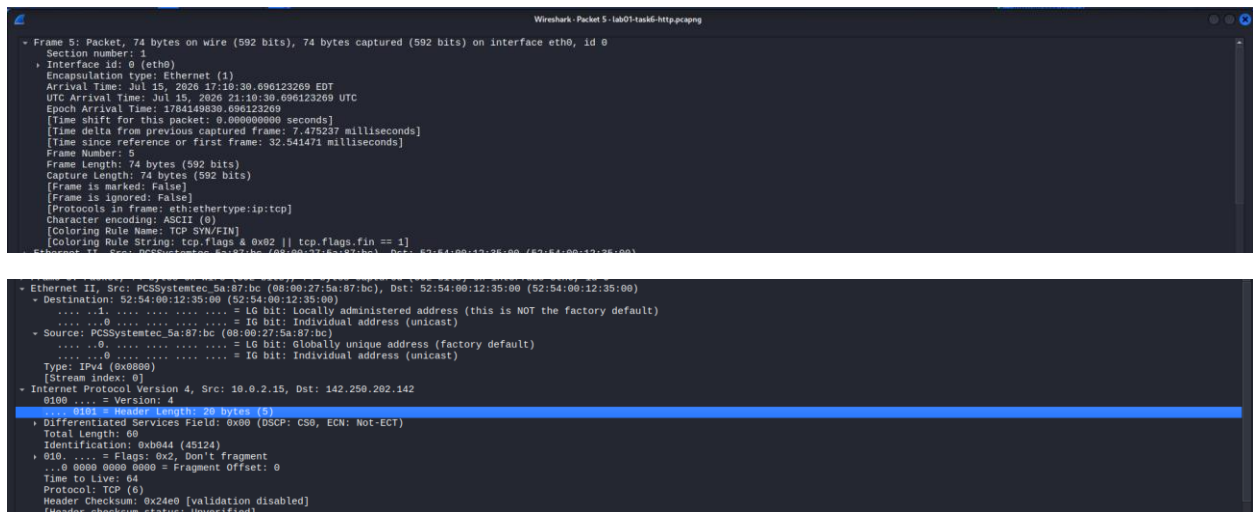
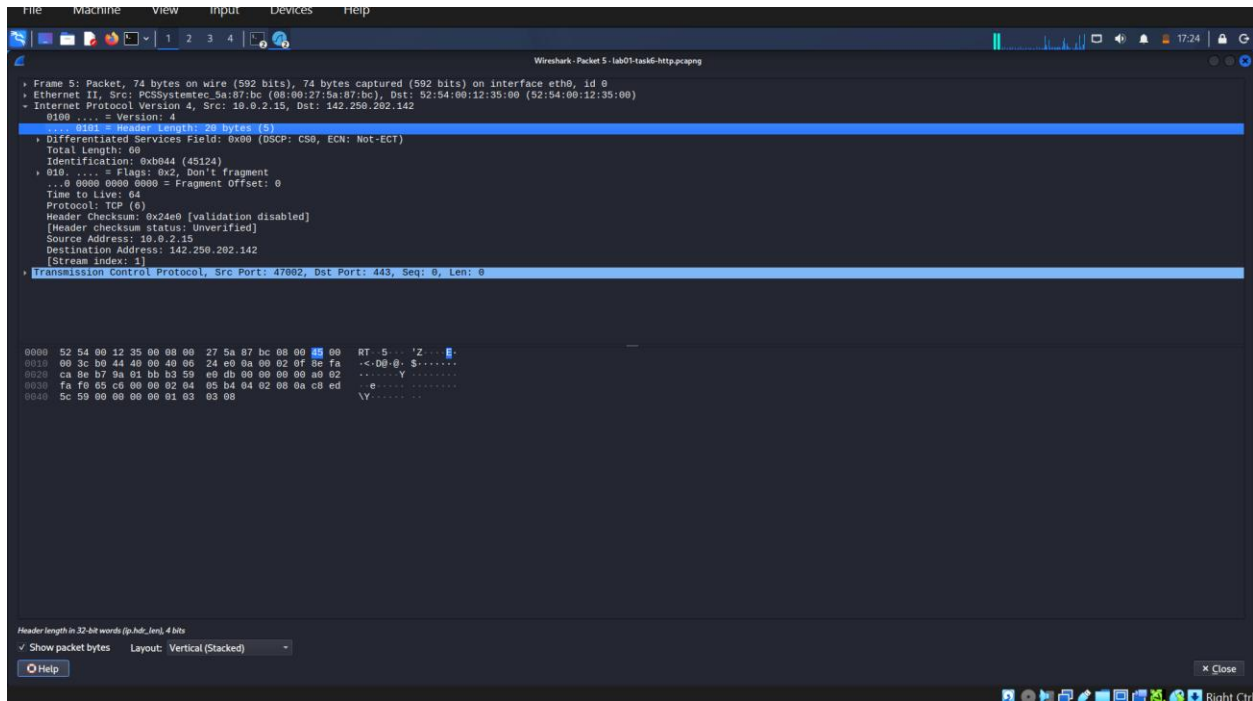
Packet 5 (the initial TCP SYN packet) from lab01-task6-http.pcapng was selected in Wireshark, and each protocol layer was expanded in the Packet Details pane.

12.3: Result:

Field	Value
Source MAC	08:00:27:5a:87:bc
Destination MAC	52:54:00:12:35:00
Source IP	10.0.2.15
Destination IP	142.250.202.142
TTL	64
Protocol	TCP (6)
Packet Length	74 bytes (592 bits)

Additional fields observed: TCP flags 0x002 (SYN), source port 47002, destination port 443, sequence number 0.

12.4: Screenshots:



```

Internet Protocol Version 4, Src: 10.0.2.15, Dst: 142.250.202.142
  0100 .... = Version: 4
  .... 0101 = Header Length: 20 bytes (5)
  Differentiated Services Field: 0x00 (DSCP: CS0, ECN: Not-ECT)
    0000 00.. = Differentiated Services Codepoint: Default (0)
    .... 00 = Explicit Congestion Notification: Not ECN-Capable Transport (0)
  Total Length: 60
  Identification: 0xb844 (45124)
  010. .... = Flags: 0x2, Don't fragment
    0. .... = Reserved bit: Not set
    1. .... = Don't fragment: Set
    0. .... = More fragments: Not set
    ... 0000 0000 0000 = Fragment Offset: 0
  Time to Live: 64
  Protocol: TCP (6)
  Header Checksum: 0x24e0 [validation disabled]
  [Header checksum status: Unverified]
  Source Address: 10.0.2.15
  Destination Address: 142.250.202.142
  [Stream index: 1]

Transmission Control Protocol, Src Port: 47062, Dst Port: 443, Seq: 0, Len: 0
  0000 52 54 00 12 35 00 00 00 27 5a 87 bc 08 00 45 00 RT 5... 'Z... E
  0010 00 3c b0 44 40 00 40 00 24 e0 0a 00 02 0f 8e fa < D0 0: $
  0020 ca 8e b7 9a 01 bb b3 59 e0 db 00 00 00 a0 02 ..... Y
  0030 fa f0 c6 00 00 02 04 05 b4 04 02 08 0a c8 ed ..e.....
  0040 5c 59 00 00 00 01 03 03 00 ..Y.....

Transmission Control Protocol, Src Port: 47062, Dst Port: 443, Seq: 0, Len: 0
  Source Port: 47062
  Destination Port: 443
  [Stream index: 0]
  [Stream Packet Number: 1]
  [Conversation completeness: Complete, WITH_DATA (31)]
  [TCP Segment Len: 0]
  Sequence Number: 0 (relative sequence number)
  Sequence Number (raw): 3609021931
  [Next Sequence Number: 1 (relative sequence number)]
  Acknowledgment Number: 0
  Acknowledgment number (raw): 0
  1010 .... = Header Length: 40 bytes (10)
  Flags: 0x002 (SYN)
  Window: 64240
  [Calculated window size: 64240]
  Checksum: 0x65c6 [unverified]
  [Checksum Status: Unverified]
  Urgent Pointer: 0
  Options: (20 bytes), Maximum segment size, SACK permitted, Timestamps, No-Operation (NOP), Window scale
  [Timestamps]
  [Client Contiguous Streams: 1]
  [Server Contiguous Streams: 1]
  0000 52 54 00 12 35 00 00 00 27 5a 87 bc 08 00 45 00 RT 5... 'Z... E
  0010 00 3c b0 44 40 00 40 00 24 e0 0a 00 02 0f 8e fa < D0 0: $
  0020 ca 8e b7 9a 01 bb b3 59 e0 db 00 00 00 a0 02 ..... Y
  0030 fa f0 c6 00 00 02 04 05 b4 04 02 08 0a c8 ed ..e.....
  0040 5c 59 00 00 00 01 03 03 00 ..Y.....

```

12.5: Analysis:

The destination MAC address (52:54:00:12:35:00) belongs to the NAT Network gateway, not the final destination server, while the destination IP (142.250.202.142) identifies the true end-to-end target. This confirms that MAC addressing operates only at the local network segment (Layer 2), while IP addressing carries traffic across the wider network to its true destination (Layer 3), with routing devices rewriting the MAC address at each hop while preserving the original IP addressing.

12.6: Conclusion:

A single packet, when fully expanded, reveals both the immediate delivery path (MAC) and the ultimate communication endpoints (IP), along with protocol-specific state information (TCP flags, sequence numbers), demonstrating why packet-level inspection provides far more investigative detail than summary logs alone.

13. Task 8: TCP Stream Reconstruction

13.1: Objective:

Reconstruct a full TCP conversation to observe the complete exchange between client and server.

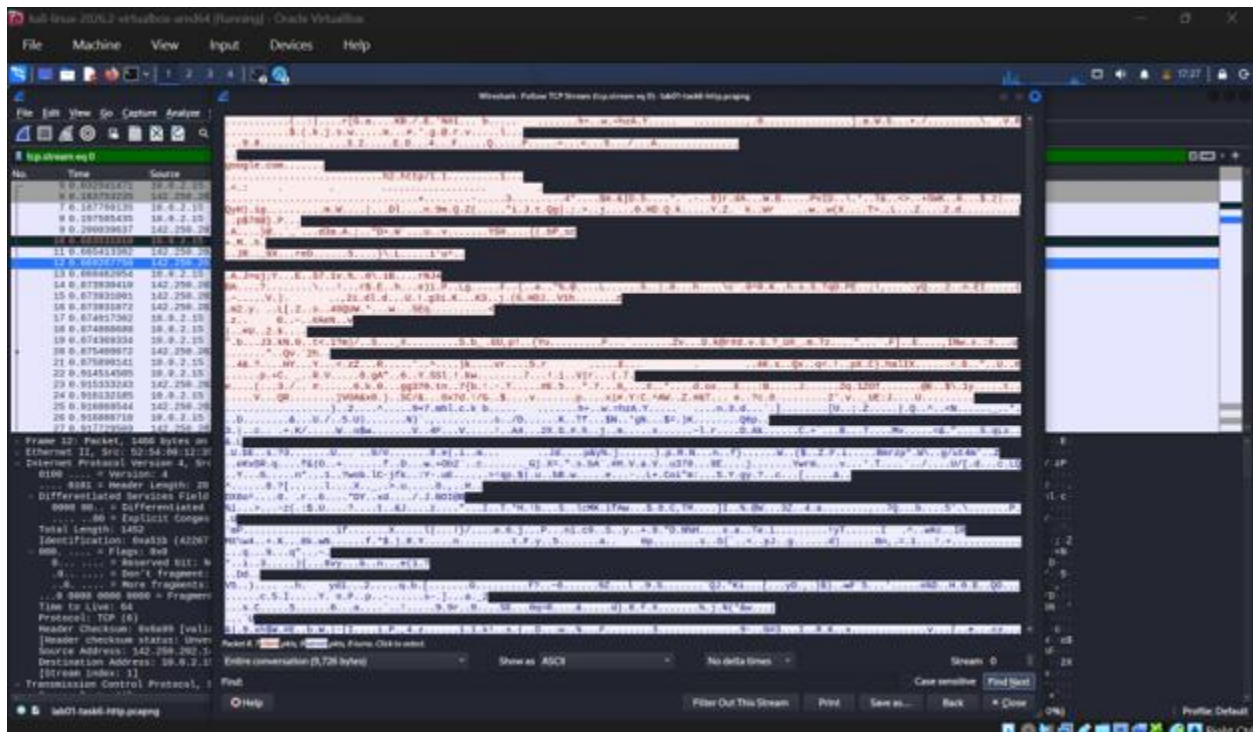
13.2: Procedure:

In lab01-task6-http.pcapng, packet 5 was right-clicked and reconstructed via Wireshark's Follow → TCP Stream feature.

13.3: Result:

The reconstructed stream showed 8 client packets and 9 server packets across 6 turns, totaling 9,726 bytes. The overwhelming majority of the stream was unreadable encrypted binary data, with one notable exception near the start of the stream: the plaintext strings google.com and http/1.1 were visible.

13.4: Screenshots:



13.5: Analysis:

The two readable fragments originate from the TLS Client Hello, specifically the SNI (Server Name Indication) field and ALPN (Application-Layer Protocol Negotiation) extension. These fields are transmitted in plain text by design, before encryption keys are established, because the server must know which certificate to present before the TLS handshake can complete. Everything transmitted after the handshake completes was fully encrypted and unreadable, providing direct, visual confirmation that HTTPS was functioning as intended for this connection.

13.6: Conclusion:

Stream reconstruction is a powerful technique for reviewing an entire conversation in sequence rather than packet-by-packet, and this task specifically demonstrates the boundary between what HTTPS exposes (connection setup metadata) and what it protects (actual content).

14. Task 9: Evidence Export

14.1: Objective:

Preserve all captured traffic as reusable evidence files.

14.2: Procedure:

Rather than producing a single combined capture file as originally specified, a separate .pcapng file and corresponding tshark text export were produced for each task (Tasks 3, 4, 5, and 6), consistent with the Evidence Collection Policy established in Section 5.

14.3: Result:

Four evidence pairs were produced:

- lab01-task3-traffic-generation.pcapng / .txt
- lab01-task4-icmp-only.txt (filtered export from Task 3's capture)
- lab01-task5-dns.pcapng / .txt
- lab01-task6-http.pcapng / .txt

14.4: Analysis:

Producing per-task evidence files, rather than one combined capture, improves traceability: each file can be referenced independently against the specific task and finding it supports, without requiring a reader to apply filters to locate the relevant packets within a larger, mixed-traffic file.

14.5: Conclusion:

This approach was adopted deliberately as an improvement over the original single-file export instruction, prioritizing clarity and reproducibility of evidence over strict adherence to the lab's original deliverable format.

15. Overall Comparative Summary Table

Task	Protocol(s)	Primary Tool	Key Evidence Collected	Key Outcome
Interface Selection	Ethernet	ip a, Wireshark	Interface verification, screenshot	Confirmed eth0 as the correct capture interface.

Baseline Capture	Background Traffic	Wireshark	.pcapng, screenshot	Observed that network traffic exists even when the system is idle.
Traffic Generation	ARP, DNS, ICMP	ping	.pcapng, tshark export	Demonstrated the layered dependency of ARP, DNS, and ICMP during a simple network operation.
ICMP Analysis	ICMP	Wireshark	Filtered capture, tshark output	Verified Echo Request/Reply behavior and demonstrated non-destructive packet filtering.
DNS Analysis	DNS, UDP	dig	.pcapng, tshark export	Examined hostname resolution and identified the DNS server used by the NAT Network.
HTTPS/TLS Capture	TCP, TLS	curl	.pcapng, Wireshark, tshark	Observed the TCP three-way handshake, TLS negotiation, encrypted application data, and connection teardown.
Packet Inspection	Ethernet, IP, TCP	Wireshark	Expanded packet details	Examined packet headers and distinguished Layer 2 (MAC) from Layer 3 (IP) addressing.
TCP Stream Reconstruction	TCP, TLS	Wireshark	Follow TCP Stream	Reconstructed the client-server conversation and identified the boundary between visible TLS metadata and encrypted application data.
Evidence Export	All	Wireshark, tshark	.pcapng and .txt files	Preserved reproducible evidence for every experiment using both graphical and command-line analysis.

16. Overall Conclusion

This lab established the foundational skills required for packet-level network analysis by capturing and examining real network traffic generated through common networking tools. Rather than treating Wireshark as a packet viewer alone, the lab emphasized evidence-based analysis through the preservation of original packet captures, `tshark` exports, and annotated screenshots for every experiment.

Across the nine tasks, traffic generated by ICMP, DNS, TCP, and TLS was examined from initial packet capture through detailed protocol analysis and conversation reconstruction. The exercises demonstrated how seemingly simple network operations rely on multiple protocols working together, while reinforcing concepts such as protocol layering, packet headers, address resolution, transport-layer communication, encryption, and evidence preservation.

The knowledge and workflow developed in this lab provide the analytical foundation for subsequent networking and cybersecurity labs. Future exercises involving network reconnaissance, packet analysis, vulnerability assessment, troubleshooting, and incident investigation will build upon the ability to interpret packet-level evidence and understand network behavior from the perspective of the packets themselves.

17. References

- Wireshark Foundation. *Wireshark User's Guide*. Available at: https://www.wireshark.org/docs/wsug_html_chunked/
- Wireshark Foundation. *Display Filter Reference*. Available at: <https://www.wireshark.org/docs/dfref/>
- Wireshark Foundation. *tshark Manual Page*. Available at: <https://www.wireshark.org/docs/man-pages/tshark.html>
- The Internet Engineering Task Force (IETF). *RFC 791: Internet Protocol*. September 1981. Available at: <https://www.rfc-editor.org/rfc/rfc791>
- The Internet Engineering Task Force (IETF). *RFC 792: Internet Control Message Protocol (ICMP)*. September 1981. Available at: <https://www.rfc-editor.org/rfc/rfc792>
- The Internet Engineering Task Force (IETF). *RFC 793: Transmission Control Protocol (TCP)*. September 1981. Available at: <https://www.rfc-editor.org/rfc/rfc793>
- The Internet Engineering Task Force (IETF). *RFC 1034: Domain Names - Concepts and Facilities*. November 1987. Available at: <https://www.rfc-editor.org/rfc/rfc1034>

- The Internet Engineering Task Force (IETF). *RFC 1035: Domain Names - Implementation and Specification*. November 1987. Available at: <https://www.rfc-editor.org/rfc/rfc1035>
- Oracle. *Oracle VM VirtualBox User Manual*. Available at: <https://www.virtualbox.org/manual/>